OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 4

REPRESENTING DATA IN A COMPUTER

SUGGESTED TIME: 3 PERIODS

IN THIS CHAPTER

This chapter explores how computers encode non-integer values using fixed-point and floating-point formats, covering binary fractions, IEEE754 standards, conversion techniques, and the inevitable precision limits. Mastery of these concepts is essential for the GCE Advanced Level Computer Science examination and for understanding real-world digital systems.

- Explain the difference between fixed-point and floating-point representation of fractional numbers.
- Convert a decimal fraction to its binary fixed-point representation with a given scaling factor.
- Encode and decode a real number in IEEE754 single-precision (32-bit) and double-precision (64-bit) formats.
- Perform normalization of a binary fraction and determine the exponent bias.
- Analyse the effects of rounding, overflow, underflow, and special values (NaN, infinity) on computation accuracy.
- Design a simple algorithm to convert between decimal, fixed-point, and floating-point representations.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Fixed-Point Representation of Fractional Numbers
2. Floating-Point Representation (IEEE754)
3. Conversion Between Representations and Precision Analysis
4. Arithmetic, Exceptions, and Real-World Implications
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- Explain the difference between fixed-point and floating-point representation of fractional numbers.
- Convert a decimal fraction to its binary fixed-point representation with a given scaling factor.
- Encode and decode a real number in IEEE754 single-precision (32-bit) and double-precision (64-bit) formats.
- Perform normalization of a binary fraction and determine the exponent bias.
- Analyse the effects of rounding, overflow, underflow, and special values (NaN, infinity) on computation accuracy.
- Design a simple algorithm to convert between decimal, fixed-point, and floating-point representations.
- Evaluate the trade-offs between range, precision, and hardware complexity when choosing a representation for a given application.

PREREQUISITES

- Binary integer representation (two's complement, sign-magnitude) from Lower Sixth.
- Basic arithmetic in base 2 (addition, subtraction, multiplication).
- Understanding of scientific notation in decimal (mantissa × 10^exponent).

- Concept of bits, bytes, and word size (16, 32, 64 bits).
- Familiarity with the notion of rounding and truncation errors.

1 FIXED-POINT REPRESENTATION OF FRACTIONAL NUMBERS

Imagine a mobile money agent in Douala who must split a 10 000 FCFA transfer equally among three friends. The phone screen shows 3333.33 FCFA, but the underlying system stores the amount as a binary pattern. How does the computer represent that fractional part, and why might the total not add up exactly to 10 000 FCFA? This chapter reveals the hidden world of fractional number representation that makes such everyday transactions possible. The first and oldest answer to this problem is the **fixed-point** representation: simple, fast, and still used today in bank software, embedded controllers and audio chips.

1.1 From Integers to Fractions: the Idea of an Implicit Binary Point

EXPERIMENT **Observing the problem**

Write the number 13.625 in ordinary decimal notation. Each digit has a weight that is a power of 10: $1 \times 10^{1} + 3 \times 10^{0} + 6 \times 10^{-1} + 2 \times 10^{-2} + 5 \times 10^{-3}$. Nothing prevents us from doing exactly the same thing in base 2: digits to the right of the point simply receive negative powers of 2. The question is then purely practical: inside a machine word of 8, 16 or 32 bits, where do we place the point?

A computer word is just a row of bits; there is no physical "point" stored anywhere. The position of the binary point is a convention agreed between the programmer and the hardware. If we decide that the point always sits at the same place, we obtain the fixed-point representation.

DEFINITION **Fixed-point number**

A **fixed-point number** is a binary integer X stored in a word of N bits, interpreted with an implicit scaling factor 2^{-f}, where f is the number of fractional bits. The real value represented is

$$V = X \times 2^{-f}.$$

The position of the binary point is fixed once and for all for a given format; it is never stored in memory.

EXEMPLE **First reading of a fixed-point pattern**

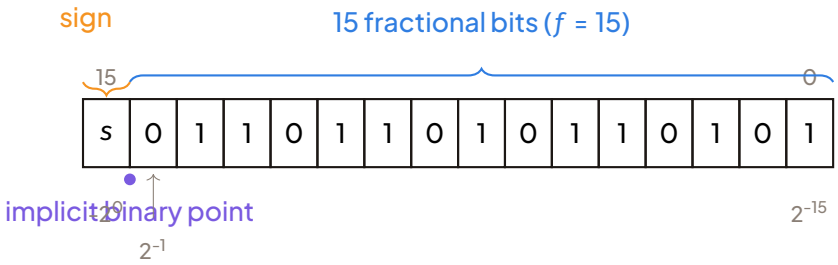
Take the 8-bit pattern 01101100_2 and declare that $f = 4$ fractional bits are used. The stored integer is $X = 108$, so

$$V = 108 \times 2^{-4} = \frac{108}{16} = 6.75.$$

The same bit pattern with $f = 2$ would mean $108 \times 2^{-2} = 27$. The bits alone say nothing: the format gives them meaning.

1.2 The Q-Format: Choosing the Scaling Factor

The standard notation for fixed-point formats is the **Q-format**. We write **Q***m*.*n* for a signed format with m integer bits (excluding the sign bit) and n fractional bits, occupying $1 + m + n$ bits in total. For example **Q15** (short for Q1.15 in some conventions, or Q0.15) denotes a 16-bit word with 1 sign bit and 15 fractional bits, widely used in digital signal processing.



A 16-bit Q15 word: one sign bit, then the implicit binary point, then 15 fractional bits with weights 2^{-1} down to 2^{-15}.

Choosing n is a trade-off: more fractional bits give finer resolution (smaller step between two consecutive representable values) but shrink the range. This trade-off is captured by the following property.

PROPERTY **Range and resolution of Q*m*.*n***

In a signed Qm.n format (1 sign bit, m integer bits, n fractional bits, two's complement):

- the **resolution** (smallest representable step) is 2^{-n};
- the **representable range** is

$$[-2^{m},\ 2^{m} - 2^{-n}].$$

Justification (examinable). The stored integer X is an $(1 + m + n)$-bit two's-complement integer, so $-2^{m+n} \le X \le 2^{m+n} - 1$. Multiplying by the scaling factor 2^{-n} gives $-2^{m} \le V \le 2^{m} - 2^{-n}$, and the gap between two consecutive values of X is 1, hence a gap of 2^{-n} between consecutive values of V.

EXEMPLE **Range of Q15 and of Q3.4**

For Q15 ($m = 0, n = 15$): range $[-1,\ 1 - 2^{-15}] = [-1,\ 0.999969\ldots]$, resolution $2^{-15} \approx 0.0000305$.
For an 8-bit Q3.4 format: range $[-8,\ 8 - 2^{-4}] = [-8,\ 7.9375]$, resolution 0.0625.

WARNING **The point is invisible**

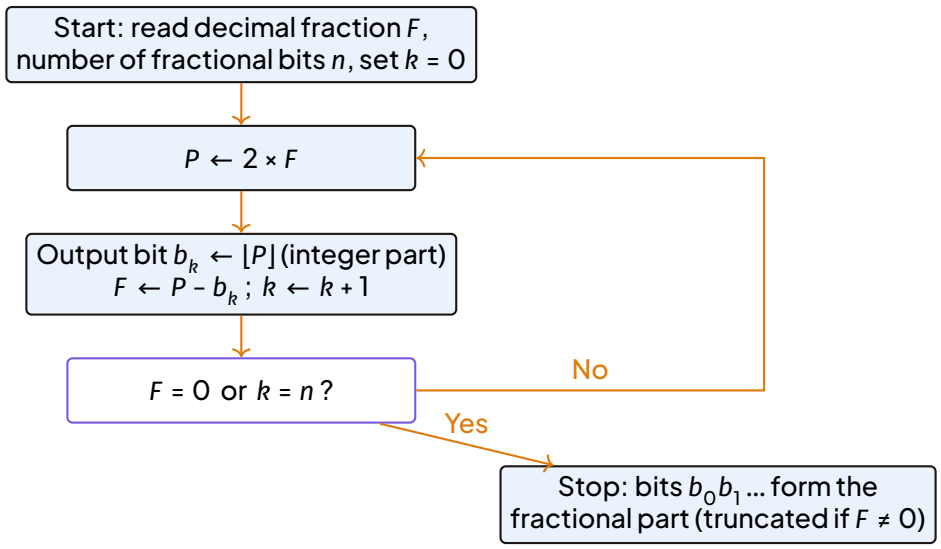
The binary point is not stored. Two programs interpreting the same memory word with different Q-formats will read different values. When you answer a GCE question, always state the assumed format before giving the value of a bit pattern.

1.3 Converting a Decimal Fraction to Binary Fixed-Point

The conversion algorithm is the mirror image of the integer "repeated division by 2" method: for the fractional part we multiply by 2 and read off the integer parts.

METHOD **Decimal fraction → binary (repeated multiplication by 2)**

1. Let F be the decimal fractional part ($0 \le F < 1$). Set the result string to empty.
2. Multiply F by 2. The integer part of the product (0 or 1) is the next binary digit, written after the point, from left to right.
3. Keep only the fractional part of the product as the new F.
4. Repeat step 2 until $F = 0$ or until you have produced the n fractional bits allowed by the target format.
5. If the process stops because the bit budget is exhausted while $F \neq 0$, the value has been truncated: a quantisation error is committed.



Flowchart of the decimal-to-binary fraction conversion by repeated multiplication by 2.

WORKED EXAMPLE 1 **Convert 0.625 and 0.1 to binary**

(a) Convert 0.625_{10} to binary.
(b) Convert 0.1_{10} to binary with 8 fractional bits, and comment.

SOLUTION

(a) Apply the method:

$$0.625 \times 2 = 1.25 \Rightarrow b_0 = 1,\ F = 0.25$$
$$0.25 \times 2 = 0.5 \Rightarrow b_1 = 0,\ F = 0.5$$
$$0.5 \times 2 = 1.0 \Rightarrow b_2 = 1,\ F = 0$$

$F = 0$, so the expansion is exact: $0.625_{10} = 0.101_2$.
(b) For 0.1:

$0.1 \times 2 = 0.2 \Rightarrow 0$ $\quad 0.2 \times 2 = 0.4 \Rightarrow 0$ $\quad 0.4 \times 2 = 0.8 \Rightarrow 0$ $\quad 0.8 \times 2 = 1.6 \Rightarrow 1$
$0.6 \times 2 = 1.2 \Rightarrow 1$ $\quad 0.2 \times 2 = 0.4 \Rightarrow 0$ $\quad \ldots$

The pattern repeats ($0.1_{10} = 0.0\overline{0011}_2$). With 8 fractional bits we keep 0.00011001_2, which equals 0.09765625, not 0.1. The quantisation error is 0.1-0.09765625 = 0.00234375. This is exactly why the Douala agent's split of 10 000 FCFA by 3 cannot be exact in binary: 1/3 is a non-terminating binary fraction, just like 0.1.

1.4 Converting Binary Fixed-Point Back to Decimal

The reverse conversion is a direct weighted sum: each bit contributes its power of two.

METHOD **Binary fixed-point → decimal**

1. Identify the format (position of the implicit point).
2. Multiply each fractional bit b_{-i} by its weight 2^{-i} and each integer bit by its weight 2^{j}.
3. Add all contributions; if the format is signed and the sign bit is 1, apply the two's-complement rule (the sign bit carries weight -2^{m}).

EXEMPLE **Weighted sum**

Interpret 0110.1100_2 as an unsigned Q4.4 value:

$$0.1100_2 = 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 0 \times 2^{-4} = 0.5 + 0.25 = 0.75,$$

so the value is 6.75. Equivalently, the stored integer is $01101100_2 = 108$ and $V = 108 \times 2^{-4} = 6.75$.

WORKED EXAMPLE 2 **Value of a signed Q15 pattern**

A 16-bit Q15 word contains the pattern `1100 0000 0000 0000`. Find the represented decimal value.

SOLUTION

The sign bit is 1; in two's complement the stored integer is $X = -2^{15} + 2^{14} = -16384$. Applying the scaling factor:

$$V = X \times 2^{-15} = -16384 \times 2^{-15} = -0.5.$$

Check with weights: the sign bit contributes $-2^{0} = -1$ and the next bit contributes $2^{-1} = 0.5$; total -0.5.

1.5 Arithmetic in Fixed-Point and Scaling Adjustments

Addition and subtraction. Two Qm.n numbers can be added or subtracted directly as integers, provided they share the same format: the common scaling factor 2^{-n} factors out:

$$(A \times 2^{-n}) + (B \times 2^{-n}) = (A + B) \times 2^{-n}.$$

No adjustment is needed, but the result may overflow the word.

Multiplication. Multiplying the stored integers multiplies the scaling factors:

$$(A \times 2^{-n}) \times (B \times 2^{-n}) = (A \times B) \times 2^{-2n}.$$

WARNING **Multiplication changes the format**

The product of two Qm.n numbers is a Q2m.2n number: it has $2n$ fractional bits and needs up to $2(1 + m + n)$ bits. To bring the result back to Qm.n you must shift right by n bits (i.e. divide the integer product by 2^n). This shift discards the n lowest bits (quantisation error) and the high bits may not fit the target word (overflow risk). Always check the magnitude of the product before truncating.

EXEMPLE **Q3.4 multiplication with rescaling**

Compute 1.5×2.25 in Q3.4. Stored integers: $1.5 \times 2^4 = 24$, $2.25 \times 2^4 = 36$. Product: $24 \times 36 = 864$, which is in Q6.8 (value $864 \times 2^{-8} = 3.375$). Shift right by 4: $864/16 = 54$, back in Q3.4: $54 \times 2^{-4} = 3.375$. Here the result is exact because no significant bit was lost.

1.6 Overflow and Quantisation Error

DEFINITION **Overflow and quantisation error in fixed-point**

- **Overflow** occurs when the exact result of an operation lies outside the representable range $[-2^m, 2^m - 2^{-n}]$; the stored bits then wrap around (in two's complement) and the value read is wrong.
- **Quantisation error** is the difference between the exact real value and the nearest representable value. With truncation it is at most one resolution step, 2^{-n}; with round-to-nearest it is at most $\frac{1}{2}\,2^{-n}$.

EXEMPLE **Detecting an overflow in Q3.4**

In Q3.4 the maximum is 7.9375. Compute $5.5 + 3.5 = 9.0$: stored integers $88 + 56 = 144$. As a signed 8-bit pattern, 144 reads as -112, i.e. $-112 \times 2^{-4} = -7.0$: the sum of two positive numbers appears negative. Detection rule for signed addition: overflow has occurred if two operands of the same sign produce a result of the opposite sign.

KEY POINTS **Key points of this section**

- A fixed-point number is an integer X read as $V = X \times 2^{-f}$; the binary point is implicit.
- Qm.n: range $[-2^m, 2^m - 2^{-n}]$, resolution 2^{-n}. More fractional bits ⇒ finer resolution, smaller range.
- Decimal fraction → binary: repeated multiplication by 2, reading integer parts left to right; many simple decimals (e.g. 0.1, 1/3) have infinite binary expansions and are truncated.
- Binary → decimal: weighted sum of the bits.
- Addition/subtraction need a common format; multiplication of two Qm.n values gives Q2m.2n and must be rescaled by a right shift of n bits.
- Always check overflow (range) and quantisation error (resolution).

DID YOU KNOW? **Exam tip**

GCE Advanced Level questions frequently give you a binary pattern and ask for its Q-format, its decimal value, or the maximum representable value of a format. Remember: the maximum of Qm.n is $2^m - 2^{-n}$ (not 2^m!), and the resolution is 2^{-n}. A quick way to answer "what is the value of this pattern?": read the pattern as an integer X (two's complement if signed) and compute $X \times 2^{-n}$.

EXERCISE 1 **Practice**

1. Give the range and resolution of a signed 12-bit Q4.7 format.
2. Convert 0.34375_{10} to binary, then write its 8-bit Q1.6 pattern (unsigned).
3. In Q15, what decimal value does the pattern `0010 0000 0000 0000` represent?
4. Two Q3.4 numbers hold 6.0 and 3.0. Compute their sum as stored integers and state whether overflow occurs.

SOLUTION **Exercise 1**

1. Range $[-2^4, 2^4 - 2^{-7}] = [-16, 15.9921875]$; resolution $2^{-7} = 0.0078125$.
2. $0.34375 \times 2 = 0.6875 \rightarrow 0$; $0.6875 \times 2 = 1.375 \rightarrow 1$; $0.375 \times 2 = 0.75 \rightarrow 0$; $0.75 \times 2 =$

1.5 → 1; 0.5 × 2 = 1.0 → 1. So $0.34375 = 0.01011_2$. In Q1.6 (7 magnitude bits): stored integer = $0.34375 \times 2^6 = 22$, pattern `00010110`.

3. Integer $X = 2^{13} = 8192$; $V = 8192 \times 2^{-15} = 0.25$.
4. 6.0 → 96, 3.0 → 48; sum = 144. Maximum Q3.4 integer is 127: 144 > 127, so overflow occurs (the 8-bit signed pattern reads -112, i.e. -7.0).

2 FLOATING-POINT REPRESENTATION (IEEE 754)

In the previous section we studied fixed-point representation, where the binary point is frozen at a fixed position. We saw that this choice is simple and fast, but it suffers from a fundamental weakness: the range of representable values is narrow. With a fixed-point format using, say, 8 integer bits and 8 fractional bits, we can represent unsigned numbers roughly between $2^{-8} \approx 0.004$ and just under 256. But a physicist in Yaoundé measuring the mass of an electron ($\approx 9.11 \times 10^{-31}$ kg) or an economist handling the national budget of Cameroon in francs CFA (thousands of billions) needs numbers that are astronomically larger or microscopically smaller. No reasonable fixed-point format can hold both at once. We need a representation in which the binary point is allowed to float — exactly what scientific notation does in decimal. This is the idea behind **floating-point** representation, standardised worldwide by the **IEEE 754** norm.

2.1 Motivation: Binary Scientific Notation and Dynamic Range

Recall how decimal scientific notation works: instead of writing 0.00000000000000000000000000000911 we write 9.11×10^{-31}. The same number is described by three pieces of information: a sign, a significand (here 9.11), and an exponent (here -31). The exponent tells us where the point goes; the significand carries the meaningful digits.

Binary scientific notation does exactly the same with powers of 2. For example:

$$13.25_{10} = 1101.01_2 = 1.10101_2 \times 2^3.$$

We simply shifted the binary point three places to the left and compensated by multiplying by 2^3. Similarly:

$$0.078125_{10} = 0.000101_2 = 1.01_2 \times 2^{-4}.$$

EXEMPLE **Binary scientific notation**

Write 5.5_{10} in normalised binary scientific notation.

- $5.5_{10} = 101.1_2$.
- Shift the point two places left: $101.1_2 = 1.011_2 \times 2^2$.
- Sign: positive; significand: 1.011_2; exponent: 2.

A floating-point format is therefore just a standardised way of packing three fields into a fixed number of bits: the **sign**, the **exponent**, and the **mantissa** (the fractional part of the significand). Because the exponent can be large positive or large negative, the same 32 bits can describe both 10^{-38} and 10^{38} — a dynamic range that fixed point can never achieve.

DEFINITION **Floating-point number (IEEE 754 normalised form)**

A normalised IEEE 754 floating-point number represents the value

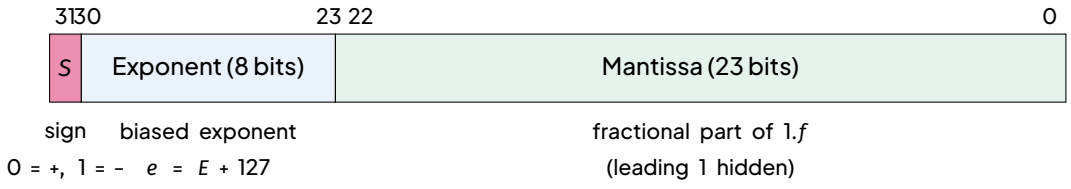
$$V = (-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{\text{exponent-bias}},$$

where sign is a single bit (0 for +, 1 for −), mantissa is the stored fractional part of the significand, exponent is the stored (biased) exponent field, and bias is a fixed constant of the format (127 in single precision, 1023 in double precision).

2.2 Structure of the IEEE 754 Formats

The IEEE 754 standard defines several formats; the two you must know for the GCE are **single precision** (binary32) and **double precision** (binary64).

Feature	Single (binary32)	Double (binary64)
Total bits	32	64
Sign bit	1	1
Exponent bits	8	11
Mantissa bits	23	52
Bias	127	1023
Approx. decimal digits	7	15–16
Approx. range (normalised)	1.2×10^{-38} to 3.4×10^{38}	2.2×10^{-308} to 1.8×10^{308}



Bit-field layout of an IEEE 754 single-precision (binary32) word.

2.3 The Biased Exponent

The actual exponent E can be negative (for small numbers) or positive (for large numbers). We could store it in two's complement, but the IEEE designers chose a cleverer trick: they store E + bias, which is always a non-negative integer, as a plain unsigned binary number.

PROPERTY **Bias representation**

The exponent field stores the value stored $= E +$ bias as an unsigned integer, so the true exponent is recovered as $E =$ stored $-$ bias. In single precision the bias is 127, so the stored exponent ranges from 0 to 255 and true exponents of normalised numbers range from −126 to +127 (the stored patterns 0 and 255 are reserved for special values). In double precision the bias is 1023. A major advantage: comparing two positive floating-point numbers can be done by comparing their bit patterns as unsigned integers, which makes hardware simpler and faster. (The statement is examinable; the justification follows directly from the definition.)

EXEMPLE **Computing a biased exponent**

The number $1.011_2 \times 2^2$ has true exponent $E = 2$. In single precision, the stored exponent is $2 + 127 = 129 = 10000001_2$. Conversely, a stored exponent field of $01111100_2 = 124$ means $E = 124 - 127 = -3$.

2.4 Normalised Numbers and the Hidden Bit

A binary number in scientific notation can always be written so that exactly one non-zero digit stands before the point — and in binary, that digit must be 1 (the only non-zero digit). So every normalised significand looks like $1.f$, where f is the fractional part. Since the leading 1 is always present, the standard simply does not store it: the 23 mantissa bits hold only f, and we say the leading 1 is **hidden** or implicit. This free trick gives one extra bit of precision: 23 stored bits deliver 24 bits of significand.

WARNING **The hidden leading 1**

The mantissa field stores only the fractional part of the significand. When decoding, you must restore the hidden leading 1: a mantissa field of 011000 … 0 means a significand of 1.011_2, not 0.011_2. Forgetting the hidden 1 is the most common exam mistake and produces answers wrong by roughly a factor of 2.

2.5 Special Values: Zero, Infinity, NaN and Subnormals

The two extreme patterns of the exponent field (all 0s and all 1s) are reserved for special values:

Exponent field	Mantissa field	Meaning
All 0s	All 0s	±0 (signed zero)
All 0s	Non-zero	Subnormal (denormalised) number: $V = (-1)^S \times 0.f \times 2^{-126}$
All 1s	All 0s	±∞ (e.g. 1.0/0.0)
All 1s	Non-zero	NaN (Not a Number, e.g. 0.0/0.0, $\sqrt{-1}$)
00000001 to 11111110	Anything	Normalised number

Subnormal numbers fill the gap between 0 and the smallest normalised number. For them the hidden bit is 0 (significand $0.f$) and the exponent is fixed at -126 (single precision). They allow gradual underflow: results that shrink below the normalised range lose precision progressively instead of collapsing abruptly to zero.

KEY POINTS **Exam tip — special bit patterns**

Learn these three patterns by heart (single precision):

- Exponent = 00000000 and mantissa = 0 … 0 ⇒ ±0.
- Exponent = 11111111 and mantissa = 0 … 0 ⇒ ±∞.
- Exponent = 11111111 and mantissa ≠ 0 … 0 ⇒ NaN.

Questions asking you to identify a special value from its bit pattern are frequent at the GCE.

2.6 Encoding a Decimal Number into binary32

METHOD **Encoding algorithm (decimal → IEEE 754 single precision)**

1. Sign: write $S = 0$ if the number is positive, $S = 1$ if negative; work with the absolute value.
2. Convert to binary: convert the integer part by successive divisions by 2 and the fractional part by successive multiplications by 2.
3. Normalise: shift the binary point to obtain $1.f \times 2^E$.
4. Bias the exponent: compute stored exponent $= E + 127$ and write it on 8 bits.
5. Mantissa: copy the bits of f (after the point), padded with zeros on the right to fill 23 bits. If f has more than 23 bits, keep the first 23 (rounding to the nearest representable value if required).
6. Pack: concatenate S (1 bit) | exponent (8 bits) | mantissa (23 bits).

WORKED EXAMPLE 3 **Encode -13.625 in IEEE 754 single precision**

Statement: Give the 32-bit IEEE 754 single-precision representation of -13.625.

SOLUTION

Step 1 — Sign. The number is negative: $S = 1$.
Step 2 — Binary conversion. $13 = 1101_2$ and $0.625 = 0.101_2$ (since $0.625 \times 2 = 1.25$, then $0.25 \times 2 = 0.5$, then $0.5 \times 2 = 1.0$, reading the integer parts 1, 0, 1). So $13.625 = 1101.101_2$.
Step 3 — Normalise. $1101.101_2 = 1.101101_2 \times 2^3$. Hence $f = 101101$ and $E = 3$.
Step 4 — Biased exponent. $3 + 127 = 130 = 10000010_2$.
Step 5 — Mantissa. Pad f to 23 bits: 1011010 00000000 00000000.
Step 6 — Pack.

$$\underbrace{1}_{S}\ \underbrace{10000010}_{\text{exponent}}\ \underbrace{10110100000000000000000}_{\text{mantissa}}$$

In hexadecimal: `C15A0000`. Check: $(-1)^1 \times 1.101101_2 \times 2^3 = -1101.101_2 = -13.625$. ✓

2.7 Decoding an IEEE 754 Pattern back to Decimal

Decoding reverses the method: split the 32 bits into the three fields, check for special patterns, restore the hidden 1, unbias the exponent, then evaluate.

WORKED EXAMPLE 4 **Decode the pattern `40400000`**

Statement: The 32-bit pattern whose hexadecimal form is `40400000` is an IEEE 754 single-precision number. Find its decimal value.

SOLUTION

Step 1 — Split the fields. `40400000` in binary is

$$\underbrace{0}_{S}\ \underbrace{10000000}_{\text{exponent}}\ \underbrace{10000000000000000000000}_{\text{mantissa}}.$$

Step 2 — Sign. $S = 0$: the number is positive.
Step 3 — Exponent. Stored exponent = $10000000_2 = 128$. True exponent $E = 128 - 127 = 1$.
Step 4 — Significand. Restore the hidden 1: significand = $1.1_2 = 1.5$.
Step 5 — Value. $V = (+1) \times 1.5 \times 2^1 = 3.0$.
So `40400000` represents exactly 3.0. ✓

WORKED EXAMPLE 5 **Decode a pattern with a longer mantissa**

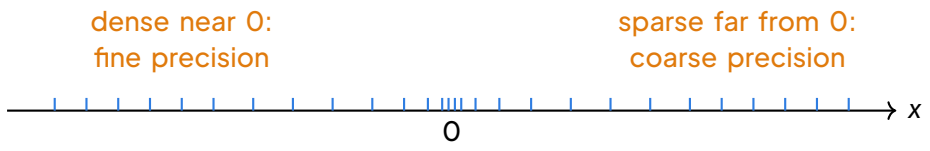
Statement: Decode the single-precision pattern `3FC00000`.

SOLUTION

Step 1 — Split. `3FC00000` = 0 01111111 10000000000000000000000.
Step 2 — Sign. $S = 0$: positive.
Step 3 — Exponent. Stored = $01111111_2 = 127$, so $E = 127 - 127 = 0$.
Step 4 — Significand. Hidden 1 restored: $1.1_2 = 1.5$.
Step 5 — Value. $V = 1.5 \times 2^0 = 1.5$. ✓

2.8 Distribution of Representable Values

A subtle but essential consequence of the floating-point design is that representable numbers are not evenly spaced. Near zero, the exponent is very negative, so consecutive values are extremely close together; for large magnitudes, the gap between two consecutive values is huge. The spacing between neighbours is proportional to the magnitude — roughly 2^{-23} times the value in single precision. Precision is therefore relative, not absolute.



Schematic distribution of representable floating-point values: clustering near zero, widening gaps at large magnitudes.

WARNING **Common mistakes with IEEE 754**

- Forgetting the hidden leading 1 when decoding (value off by a factor of about 2).
- Forgetting to subtract the bias: a stored exponent of 10000010_2 does not mean $E = 130$; the true exponent is 130 - 127 = 3.
- Believing every decimal fraction converts exactly: 0.1_{10} has an infinite binary expansion ($0.0001100110011\ldots_2$), so it is only approximated — this is why 0.1 + 0.2 ≠ 0.3 exactly in most programming languages.
- Confusing the mantissa (stored bits) with the significand ($1.f$, hidden bit included).

EXERCISE 2 **Practice**

1. Encode 6.75_{10} in IEEE 754 single precision; give the 32-bit pattern and its hexadecimal

form.

2. Decode the single-precision pattern `C1200000` to decimal.
3. Identify what each of these single-precision patterns represents: (a) `00000000`; (b) `FF800000`; (c) `7FC00000`.

SOLUTION **Exercise 1**

1. $6.75 = 110.11_2 = 1.1011_2 \times 2^2$. Sign 0; exponent $2 + 127 = 129 = 10000001_2$; mantissa 1011000 … 0. Pattern: 0 10000001 10110000000000000000000 = `40D80000`.
2. `C1200000` = 1 10000010 0100 … 0: $S = 1$, $E = 130 - 127 = 3$, significand $1.01_2 = 1.25$. Value $= -1.25 \times 2^3 = -10$.
3. (a) +0 (exponent and mantissa all zero). (b) $-\infty$ (sign 1, exponent all 1s, mantissa zero). (c) NaN (exponent all 1s, mantissa non-zero).

DID YOU KNOW? **Why IEEE 754 matters in Cameroon and beyond**

Every mobile money transaction, every weather forecast supercomputer, and every smartphone game uses IEEE 754 arithmetic. The standard, first published in 1985, ended an era in which different computers gave different answers to the same calculation. A famous cautionary tale: in 1991, a Patriot missile battery mis-tracked time because of accumulated floating-point rounding in converting tenths of a second to binary — a dramatic reminder that understanding the limits of floating-point precision is not just exam material, but real engineering.

3 CONVERSION BETWEEN REPRESENTATIONS AND PRECISION ANALYSIS

In the previous sections we mastered the IEEE 754 floating-point formats (sign bit, biased exponent, normalised significand with hidden bit) and fixed-point formats such as Q15 (scaled integers). Real systems constantly move data between these two worlds: a temperature sensor in a Yaoundé weather station delivers a Q15 value, the microcontroller converts it to `float` for computation, and the result may be converted back to fixed-point before being sent to a display. Each conversion is a potential source of error. This section gives you the rigorous tools to perform these conversions, to measure the error they introduce, and to choose the right representation for a given application — a classic GCE Advanced Level examination topic.

3.1 From Fixed-Point to Floating-Point

The idea. A fixed-point number is an integer N stored in a register together with an implicit scaling factor 2^{-f}, where f is the number of fractional bits. Its true mathematical value is $V = N \times 2^{-f}$. To convert it to IEEE 754 we must normalise the integer $|N|$ (write it as $1.m \times 2^{p}$), then adjust the exponent by combining the normalisation shift p with the fixed scaling $-f$.

METHOD **Converting a fixed-point value to IEEE 754 binary32**

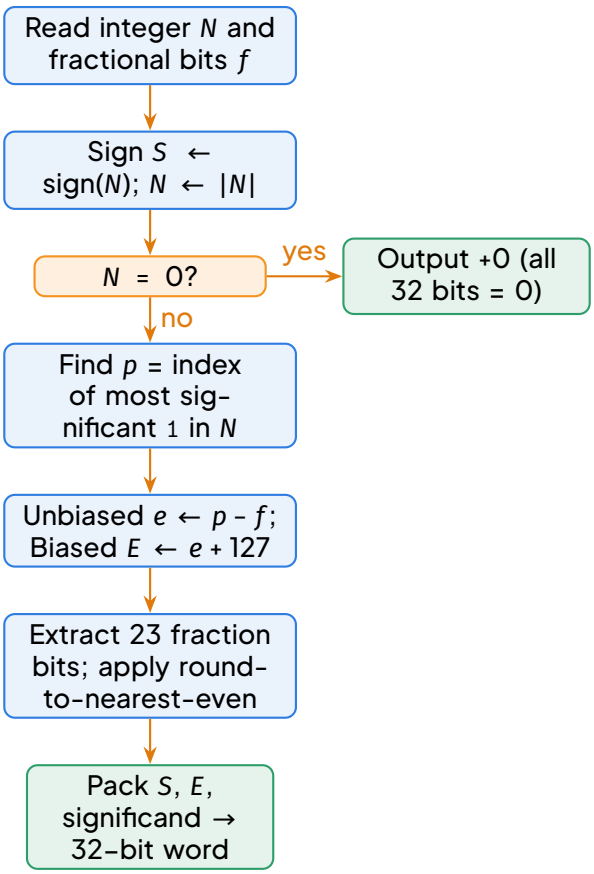
1. Sign: The sign bit S of the float is the sign of N (0 for positive, 1 for negative). Work with $|N|$ thereafter.
2. Special case: If $N = 0$, output the all-zero word (positive zero).
3. Normalise: Find the position p of the most significant 1-bit of $|N|$ (bit index 0 = LSB). Then $|N| = 1.b_{p-1}b_{p-2}\ldots b_0 \times 2^{p}$.
4. Exponent: The true value is $V = 1.b_{p-1}\ldots \times 2^{p-f}$, so the unbiased exponent is $e = p - f$. Compute the biased exponent $E = e + 127$ (bias for binary32).
5. Significand: Drop the leading 1 (hidden bit). Take the next 23 bits $b_{p-1}\ldots b_{p-23}$; if more bits remain, apply **round-to-nearest-even** (see Rounding Modes below).
6. Pack: Assemble the 32-bit word: S (1 bit) | E (8 bits) | significand (23 bits).

EXEMPLE **Q15 to binary32**

Convert the Q15 value whose stored integer is $N = 12288$ to IEEE 754 single precision. In Q15, $f = 15$, so $V = 12288 \times 2^{-15} = 0.375$.

- $N > 0 \Rightarrow S = 0$.
- $12288 = 2^{13} + 2^{12} = 11000000000000_2 = 1.1_2 \times 2^{13}$, hence $p = 13$.
- Unbiased exponent: $e = p - f = 13 - 15 = -2$. Check: $1.1_2 \times 2^{-2} = 0.011_2 = 0.375$. ✓
- Biased exponent: $E = -2 + 127 = 125 = 01111101_2$.
- Significand: hidden 1 dropped, fraction bits = 1 followed by 22 zeros → 10000000000000000000000.

Result: 0 01111101 10000000000000000000000 = 0x3EC00000.



Flowchart of the fixed-point (Qf) to IEEE 754 binary32 conversion routine.

3.2 From Floating-Point to Fixed-Point

Converting the other way is the inverse operation, but it carries a new danger: the floating-point value may be too large (overflow) or too precise (quantisation error) for the target fixed-point format.

METHOD **Converting a binary32 float to fixed-point Qf**

1. Decode: Extract sign S, biased exponent E, significand bits M. Compute unbiased exponent $e = E - 127$. Reconstruct the significand: $1.M$ (normal) or $0.M$ (subnormal).
2. Scale: Compute the exact real value $N_{real} = V \times 2^{f} = (\pm 1.M \times 2^{e}) \times 2^{f}$.
3. Round: Round N_{real} to the nearest integer N (this step introduces quantisation error; use round-to-nearest-even).
4. Range check (critical): Verify that N fits in the target signed integer width (e.g. for 16-bit Q15: $-32768 \le N \le 32767$).

5. Handle overflow: If N is out of range, saturate (clamp to nearest limit) or raise an exception — never allow silent wraparound.
6. Store: Write the 16-bit (or 32-bit) two's complement representation of N.

EXEMPLE **binary32 to Q15**

Convert the float $V = 1.101_2 \times 2^{-3} = 0.203125$ to Q15 ($f = 15$).

- $N_{\text{real}} = 0.203125 \times 2^{15} = 6656.0$ exactly $\Rightarrow N = 6656$, zero quantisation error.
- Range check: $6656 \leq 32767$ ✓. Stored Q15 word: $6656 = 0001101000000000_2$.

By contrast, $V = 0.1$ gives $N_{\text{real}} = 3276.8$, which rounds to 3277: the stored value is $3277 \times 2^{-15} = 0.1000061\ldots$, an absolute error of about 6.1×10^{-6}.

WARNING **Range checking is not optional**

A Q15 format represents values only in $[-1,\ 1 - 2^{-15}]$. Converting the float 1.5 to Q15 without a range check silently wraps the integer arithmetic (on typical two's complement hardware) and can produce a negative result. Safe embedded code (as found in mobile money terminals) always saturates: if $V \times 2^f$ exceeds the maximum, store the maximum representable integer.

3.3 Rounding Modes

When the exact value cannot be represented, the hardware must choose a nearby representable value. IEEE 754 defines four rounding modes; the default is round-to-nearest-even.

DEFINITION **Rounding modes of IEEE 754**

Let x be the exact real value and let $a < x < b$ where a and b are two consecutive representable numbers in the target format.

- **Round-to-nearest-even** (default): choose the nearer of a and b; if x is exactly halfway ($x = (a + b)/2$), choose the one whose least significant significand bit is 0 ("even").
- **Round-toward-zero** (truncation): choose the one of a, b with the smaller absolute value.
- **Round-up** (toward $+\infty$): choose b.
- **Round-down** (toward $-\infty$): choose a.

METHOD **Applying round-to-nearest-even (binary)**

1. Write the exact binary significand and mark the last bit that will be kept (the rounding position).
2. Examine the discarded bits (the guard, round, and sticky bits conceptually).
3. If the discarded part is less than half an ULP (i.e. starts with `0`), round down (keep the kept bits unchanged).
4. If the discarded part is greater than half an ULP (starts with `1` and has at least one more `1` somewhere), round up (add 1 to the last kept bit).
5. If the discarded part is exactly half an ULP (i.e. `1000…`), round to even: make the last kept bit `0` (if it is `1`, add 1; if it is `0`, leave it).

EXEMPLE **A tie case**

Suppose we keep 4 fraction bits and the exact normalised significand is $1.0101\,1000\,\ldots_2$ — the discarded part is exactly `1000…` (half ULP). The two candidates are 1.0101_2 (ends in `1`, odd) and 1.0110_2 (ends in `0`, even). Round-to-nearest-even picks 1.0110_2. Had the kept bits been 1.0110 with the same tie, we would keep 1.0110 unchanged. This symmetric rule avoids the systematic upward bias that simple "round half up" would introduce over millions of operations.

3.4 Quantisation Error, Relative Error and Machine Epsilon

Every rounding replaces the exact value x by a representable value $\hat{x}$. We measure the damage with two quantities:

$$\text{absolute error} = |x - \hat{x}|, \qquad \text{relative error} = \frac{|x - \hat{x}|}{|x|} \quad (x \neq 0).$$

DEFINITION **Machine epsilon (ε)**

The **machine epsilon** ε of a floating-point format is the smallest positive number such that $1.0 + \varepsilon \neq 1.0$ in that format. Equivalently, it is the gap (ULP) between 1.0 and the next larger representable number, i.e. $\varepsilon = 2^{-p}$ where p is the number of stored significand bits.

PROPERTY **Machine epsilon of IEEE 754 formats**

- Single precision (binary32, 23 stored bits): $\varepsilon = 2^{-23} \approx 1.19 \times 10^{-7}$.
- Double precision (binary64, 52 stored bits): $\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$.

Consequently, the relative error of rounding any normal number with round-to-nearest is at most $\varepsilon/2$ (called the unit roundoff u). For binary32, $u = 2^{-24} \approx 5.96 \times 10^{-8}$. (The derivation — the gap at 1.0 is one ULP — is instructive but not required at GCE.)

WARNING **Fixed-point has no epsilon in this sense**

In Q15 the gap between consecutive values is the constant $2^{-15} \approx 3.05 \times 10^{-5}$ everywhere in the range. In floating-point the gap grows with the magnitude: near 1000 the gap in binary32 is $1000 \times 2^{-23} \approx 1.2 \times 10^{-4}$. Floating-point keeps the relative error bounded; fixed-point keeps the absolute error bounded.

WORKED EXAMPLE 6 **Absolute and relative error of 0.1 in binary32 — classic GCE question**

Statement: The decimal number 0.1 is stored in IEEE 754 single precision as the nearest value $\hat{x} = 0.100000001490116119384765625$. Compute the absolute and relative errors, and compare the relative error with the unit roundoff $u = 2^{-24}$.

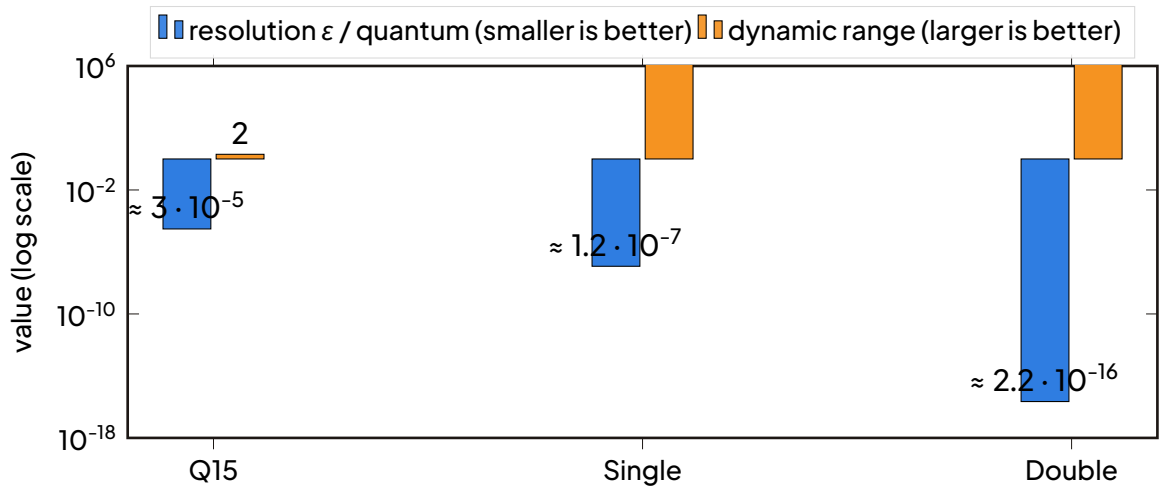
SOLUTION

Solution.

$$\text{absolute error} = |0.1 - \hat{x}| = 1.490116119384765625 \times 10^{-9} \approx 1.49 \times 10^{-9}.$$

$$\text{relative error} = \frac{1.49 \times 10^{-9}}{0.1} \approx 1.49 \times 10^{-8}.$$

The unit roundoff is $u = 2^{-24} \approx 5.96 \times 10^{-8}$. Since $1.49 \times 10^{-8} < 5.96 \times 10^{-8}$, the relative error is within the guaranteed bound: the representation of 0.1 is as good as binary32 can possibly be. The error exists because $0.1 = 0.000\overline{1100}_2$ is periodic in binary and can never be stored exactly with finitely many bits.



Comparison of Q15 fixed-point, IEEE 754 single and double precision: resolution (machine epsilon or quantum) and dynamic range (ratio of largest to smallest positive normal value, log scale).

3.5 Propagation of Rounding Errors

A single rounding is tiny; a sequence of roundings can accumulate into a disaster. Two mechanisms dominate.

Accumulation (swallowing). Suppose we sum 10^8 terms each equal to 0.1 in binary32. The true sum is 10^7. Each addition rounds the running total, and once the running total is large, adding 0.1 changes it by less than the gap between representable numbers near it — the small terms are partly or wholly swallowed. The accumulated error can reach several units in the final result. A classical remedy is **Kahan compensated summation**, which keeps a separate variable tracking the lost low-order bits and restores them at each step; simply sorting the terms from smallest to largest before summing also helps.

Catastrophic cancellation. When two nearly equal numbers are subtracted, their common leading bits cancel and the result is made of the low-order bits — precisely where the rounding errors live.

WARNING **Catastrophic cancellation**

Subtracting two nearly equal floating-point numbers causes **catastrophic cancellation**: the result may keep almost no significant digits. Example with 8 significant decimal digits: $x = 1.2345678$, $y = 1.2345677$, both correctly rounded. The difference $x - y = 1.0 \times 10^{-7}$ has only one significant digit; if x and y were themselves rounded results, that single digit may be pure noise. Rewrite formulas to avoid such subtractions (e.g. use $\sqrt{x+1} - \sqrt{x} = \dfrac{1}{\sqrt{x+1}+\sqrt{x}}$ for large x).

EXEMPLE **Swallowing in practice**

In binary32, compute $10^8 + 1$ (exact arithmetic = 100000001). $10^8 \approx 2^{26.575}$, so its exponent is 26. The gap (ULP) near 10^8 is $2^{26-23} = 2^3 = 8$. Therefore $10^8 + 1$ rounds back to 10^8: the 1 vanishes completely. Adding 1 a hundred million times in a loop starting from 10^8 still leaves 10^8 — whereas adding the ones first (summing to 10^8) and then adding 10^8 gives the correct answer 2×10^8. Order of operations matters critically.

3.6 Choosing Between Fixed and Floating Point: Guidelines

METHOD **Decision guide for embedded systems**

1. Hardware support: Does the processor have a floating-point unit (FPU)? Low-cost microcontrollers used in many Cameroonian devices (solar charge controllers, basic POS terminals) do not; floating-point is then emulated in software and is 10–100 times slower.
2. Range of values: If all quantities stay within a known, narrow range (e.g. a battery voltage between 10.0 and 14.5 V), fixed-point is ideal and exact to its quantum.
3. Monetary values: For mobile money terminals (MTN MoMo, Orange Money), never use binary floating-point for amounts in FCFA: 0.1 is inexact in binary, and rounding errors on balances are unacceptable. Use integers counting the smallest unit (i.e. a fixed-point convention: Q0 on francs, or centimes where applicable).
4. Wide dynamic range: If values span many orders of magnitude (scientific computation, audio signal processing before scaling), choose floating-point — double precision if memory allows.
5. Error budget: Compute the worst-case absolute error (fixed-point: half the quantum per operation) or relative error (floating-point: u per operation) and check it against the application's tolerance.

DID YOU KNOW? **A famous and costly rounding bug**

In 1991, during the Gulf War, a Patriot missile battery failed to intercept an incoming missile because its internal clock accumulated time in tenths of a second stored with a 24-bit binary approximation of 0.1 — exactly the inexact value you studied above. After about 100 hours of continuous operation the accumulated error was roughly 0.34 s, enough for the system to look for the target in the wrong place. The lesson for every engineer: understand the error behaviour of your number representation before lives — or money — depend on it.

EXPERIMENT **Integration Activity: Designing a Data Acquisition Chain**

A weather station in Buea uses a 12-bit ADC (0–4095) measuring 0–3.3 V. The voltage represents temperature: $T = (V - 0.5) \times 100$ °C. The microcontroller (Cortex-M0, no FPU) must transmit T over a CAN bus as a 16-bit signed integer in Q8 format (8 fractional bits).

1. Determine the fixed-point scaling for the ADC raw value to get voltage in Q12.
2. Derive the integer arithmetic sequence (using 32-bit temporaries) to compute the Q8 temperature value directly from the ADC raw value, avoiding floating-point entirely.
3. What is the maximum absolute error introduced by the Q8 quantisation? Is it acceptable for a display resolution of 0.1 °C?

Guidance: This activity combines fixed-point scaling, range analysis, and error budgeting — exactly the skills examined in GCE Practical / Project work.

KEY POINTS **Key points of this section**

- Fixed → float: normalise the integer, exponent $e = p - f$, bias it, round the significand.
- Float → fixed: multiply by 2^f, round to an integer, and always check the range (saturate on overflow).
- Default rounding is **round-to-nearest-even**; ties are resolved by making the last kept bit even.
- Machine epsilon: $\varepsilon = 2^{-23} \approx 1.19 \times 10^{-7}$ (single), $2^{-52} \approx 2.22 \times 10^{-16}$ (double); relative rounding error $\leq \varepsilon/2 = u$.
- Errors accumulate in long sums (use Kahan or sort terms) and explode under **catastrophic cancellation**.
- Money and narrow-range control: fixed-point/integer. Wide dynamic range: floating-point.

EXERCISE 3 **Precision analysis**

1. A sensor delivers temperatures between -40 and +85 °C. A designer stores them in Q8 format (8 fractional bits, 16-bit signed word). What is the quantum? Is the range sufficient?
2. The value 0.2 is stored in binary32 as 0.200000002980232238769531250. Compute the absolute and relative errors and verify that the relative error is below 2^{-24}.

3. Explain why the loop "`s = 0; repeat 10 times: s = s + 0.1`" in binary32 does not produce exactly 1.0, and state which rounding mode the additions use by default.

SOLUTION **Exercise 1**

1. Quantum = 2^{-8} = 0.00390625 °C. Range of signed 16-bit Q8: $[-2^7, 2^7 - 2^{-8}]$ = $[-128, 127.996]$, which comfortably covers $[-40, 85]$. The format is suitable.
2. Absolute error = $2.98023223876953125 \times 10^{-9} \approx 2.98 \times 10^{-9}$. Relative error = $2.98 \times 10^{-9} / 0.2 \approx 1.49 \times 10^{-8}$. Since $2^{-24} \approx 5.96 \times 10^{-8}$ and $1.49 \times 10^{-8} < 5.96 \times 10^{-8}$, the bound is respected.
3. Each 0.1 is stored with error $\approx 1.49 \times 10^{-9}$, and each addition rounds the partial sum using the default round-to-nearest-even mode. The ten small errors do not cancel exactly, so the final sum differs from 1.0 by a few units in the last place; the test `s == 1.0` fails, which is why floating-point values must be compared with a tolerance, e.g. $|s - 1.0| < 10^{-6}$.

4 ARITHMETIC, EXCEPTIONS, AND REAL WORLD IMPLICATIONS

In the previous sections we mastered the conversion between fixed-point, floating-point and decimal representations, and we analysed how precision is limited by the number of bits allocated to the mantissa. We now turn to what happens when we actually compute with these representations. The IEEE 754 standard does not only define bit layouts; it also specifies exactly how arithmetic operations must be performed, how exceptional situations are signalled, and how special values (infinities, NaNs) behave in calculations. Understanding these rules is essential for writing reliable numerical software — and for avoiding costly mistakes in real-world applications such as financial systems in Cameroon.

4.1 Floating-Point Addition and Subtraction

Adding two floating-point numbers is not as simple as adding their bit patterns. Because the numbers may have different exponents, we must first align them, then add the mantissas, then normalize the result, and finally round to the target precision. The same steps apply to subtraction (adding the negated second operand).

METHOD **Floating-Point Addition / Subtraction (IEEE 754 binary32)**

1. Unpack: Extract sign s, exponent E (biased), and mantissa M (with implicit leading 1 for normals, 0 for subnormals) for both operands A and B.
2. Align exponents: Let $\Delta E = E_A - E_B$. If $\Delta E > 0$, shift M_B right by ΔE bits (inserting guard, round, and sticky bits); if $\Delta E < 0$, shift M_A right by $|\Delta E|$ bits. The larger exponent becomes the tentative result exponent E_R.
3. Add/Subtract mantissas: Perform signed addition/subtraction on the aligned mantissas (including the implicit bit and guard bits). The result mantissa M_R may have a leading bit at position 1 (normal) or 0 (requiring normalization).
4. Normalize: If M_R has a leading 1 at bit 1 (i.e., value ≥ 2), shift right by 1 and increment E_R. If the leading 1 is at bit 0 (value < 1 but $\geq 1/2$ for normals), shift left until the leading 1 reaches bit 1, decrementing E_R accordingly. For subnormals, the leading bit may be 0.
5. Round: Apply the current rounding mode (default: round to nearest, ties to even) using the guard, round, and sticky bits. Rounding may cause a carry into the leading bit, requiring a final renormalization step.
6. Pack: Assemble the sign, biased exponent, and the 23-bit mantissa (dropping the implicit leading 1). Handle special cases: overflow → $\pm\infty$, underflow → subnormal or ± 0.

EXEMPLE **Tracing binary32 addition:** $1.5 + 2^{-24}$

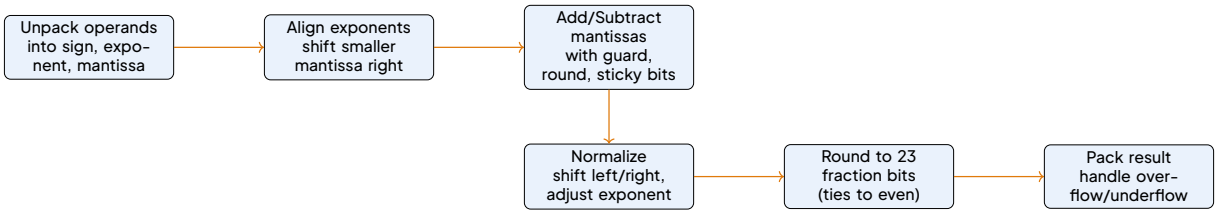
Let $A = 1.5 = 1.1_2 \times 2^0$ and $B = 2^{-24} = 1.0_2 \times 2^{-24}$.

1. Unpack: A: $s = 0$, $E = 127$ (biased), $M_A = 1.1_2$ (implicit 1 + fraction .1). B: $s = 0$, $E = 103$, $M_B = 1.0_2$.
2. Align: $\Delta E = 24$. Shift M_B right by 24 bits: $M'_B = 0.000\ldots01_2$ (1 at the 24th fractional bit). Guard bit = 0, round bit = 0, sticky bit = 0 (since no bits beyond).
3. Add: $M_A + M'_B = 1.100\ldots01_2$ (24 zeros then 1). No carry beyond the leading 1.
4. Normalize: Leading 1 already at bit 1, exponent unchanged ($E_R = 127$).
5. Round: The bits beyond the 23-bit fraction are 0 (guard), 0 (round), 0 (sticky) → round down (truncate). Result mantissa = 1.1_2 exactly.
6. Result: 1.5 — the tiny addition was lost because it fell below the precision of binary32 (machine epsilon $\approx 2^{-23}$).

EXEMPLE **Tracing binary32 subtraction:** $1.0 \times 2^5 - 1.5 \times 2^2$

Let $A = 1.0_2 \times 2^5$ ($E_A = 132$, $M_A = 1.0$) and $B = 1.5 = 1.1_2 \times 2^2$ ($E_B = 129$, $M_B = 1.1$).

1. Align: $\Delta E = 3$. Shift M_B right by 3: $M'_B = 0.0011_2$ (guard=0, round=0, sticky=0).
2. Subtract: $M_A - M'_B = 1.0000_2 - 0.0011_2 = 0.1101_2$.
3. Normalize: Leading 1 at bit 0 → shift left by 1: $M_R = 1.101_2$, $E_R = 131$.
4. Round: Extra bits all zero → exact.
5. Result: $1.101_2 \times 2^4 = 1.625 \times 16 = 26.0$ (exact).



Sequence of steps in IEEE 754 binary32 addition/subtraction.

4.2 Floating-Point Multiplication and Division

Multiplication and division are conceptually simpler because they do not require exponent alignment. However, they produce a mantissa with up to twice the number of bits, so rounding is critical.

METHOD **Floating-Point Multiplication (IEEE 754 binary32)**

1. Unpack: Extract s_A, E_A, M_A and s_B, E_B, M_B.
2. Sign: $s_R = s_A \oplus s_B$ (XOR).
3. Exponent: $E_R = E_A + E_B - 127$ (bias adjustment). Check for overflow/underflow later.
4. Mantissa: Multiply the two 24-bit mantissas (including implicit leading 1) → 48-bit product P.
5. Normalize: P is in $[1, 4)$. If $P \geq 2$, shift right by 1 and increment E_R.
6. Round: Round the 48-bit mantissa to 23 fraction bits using guard/round/sticky bits.
7. Pack: Assemble result; handle special cases (± 0, $\pm\infty$, NaN).

METHOD **Floating-Point Division (IEEE 754 binary32)**

1. Unpack, Sign, Exponent: $s_R = s_A \oplus s_B$; $E_R = E_A - E_B + 127$.
2. Mantissa: Compute $Q = M_A / M_B$ to sufficient precision (at least 26 bits: 24 + guard + round + sticky).
3. Normalize: Q is in $[0.5, 2)$. If $Q < 1$, shift left by 1 and decrement E_R.
4. Round and Pack: As in multiplication.

WORKED EXAMPLE 7 **Multiplication trace: 1.5 × 0.5 in binary32**

$A = 1.5 = 1.1_2 \times 2^0$ ($E_A = 127, M_A = 1.1_2$). $B = 0.5 = 1.0_2 \times 2^{-1}$ ($E_B = 126, M_B = 1.0_2$).

1. Sign: $0 \oplus 0 = 0$.
2. Exponent: $E_R = 127 + 126 - 127 = 126$ (biased) → true exponent -1.
3. Mantissa product: $1.1_2 \times 1.0_2 = 1.1_2$ (exactly 1.5 in decimal, but $1.5 \times 0.5 = 0.75 = 1.1_2 \times 2^{-1}$). The product is 1.1_2, already normalized (leading 1 at bit 1). No shift needed.
4. Round: No extra bits → exact.
5. Result: $s = 0$, $E = 126$, fraction = $.1_2$ → binary32 representation of 0.75.

4.3 IEEE 754 Exception Flags and Special Values

The IEEE 754 standard defines five exception flags (sticky bits) that are set by hardware whenever the corresponding condition occurs during an operation. They remain set until explicitly cleared by software. These flags allow programs to detect exceptional conditions without trapping (which would be slow).

DEFINITION **IEEE 754 Exception Flags**

- Invalid Operation: Set when an operation has no mathematically defined result (e.g., $\sqrt{-1}$, $0/0$, $\infty - \infty$, $0 \times \infty$, comparison with NaN). Result: **NaN** (Not a Number).
- Division by Zero: Set when a finite non-zero number is divided by zero. Result: $\pm\infty$ (sign according to operands).
- Overflow: Set when the rounded result exceeds the largest finite representable number (in magnitude). Result: $\pm\infty$ (default rounding) or largest finite (round toward zero).
- Underflow: Set when a non-zero result is too small to be represented as a normalized

number (i.e., exponent would be < −126 for binary32). Result: subnormal or ±0; the inexact flag is also set if rounding occurred.

- Inexact: Set whenever the rounded result differs from the infinitely precise result (i.e., almost every floating-point operation except exact additions/multiplications of small integers).

PROPERTY **Arithmetic with Special Values**

The following rules govern operations involving infinities and NaNs (consistent with limits of real analysis where possible):

$$\begin{aligned} \infty + \infty &= \infty \quad \text{(same sign)} \\ \infty - \infty &= \text{NaN} \quad \text{(invalid operation)} \\ 0 \times \infty &= \text{NaN} \quad \text{(invalid operation)} \\ x / \pm\infty &= \pm 0 \quad (x \text{ finite}) \\ \pm\infty / x &= \pm\infty \quad (x \text{ finite, non-zero}) \\ \text{Any operation with NaN} &= \text{NaN} \quad \text{(propagates)} \\ \text{Comparison with NaN} &= \text{False (except } \neq \text{ which is True)} \end{aligned}$$

WARNING **Never test floating-point equality for money**

Because of rounding errors, two floating-point values that should be mathematically equal (e.g., 0.1 + 0.2 vs 0.3) will often differ in their least significant bits. Never write `if (price == expected)` for monetary amounts. Always use integer representation of the smallest currency unit (centimes) or a decimal library.

4.4 Case Study: VAT Calculation on a Cameroonian E-Commerce Platform

Consider a Cameroonian e-commerce platform that sells 1000 items, each priced at 1234 FCFA. The VAT rate is 19.25 %. The platform stores prices as `float` (binary32) and computes the total VAT-inclusive revenue in a loop. We analyse the rounding impact.

EXPERIMENT **VAT Calculation: binary32 vs Integer Centimes**

Scenario: 1000 items at 1234 FCFA each. VAT = 19.25 %. Total revenue = 1000 × 1234 × 1.1925 = 1 471 545 FCFA exactly (since 1234 × 1.1925 = 1471.545, times 1000 = 1 471 545). Method A (binary32 loop):

- `float price = 1234.0f;`
- `float vatRate = 1.1925f;`
- `float total = 0.0f;`
- `for (int i = 0; i < 1000; i++) total += price * vatRate;`

Method B (integer centimes):

- `long priceCentimes = 123400;` // 1234 FCFA = 123400 centimes
- `long vatNum = 11925;` // 1.1925 = 11925/10000
- `long vatDen = 10000;`
- `long totalCentimes = 0;`
- `for (int i = 0; i < 1000; i++) totalCentimes += priceCentimes * vatNum / vatDen;`

Results (simulated):

- Exact total: 1 471 545 FCFA = 147 154 500 centimes.
- Method A (binary32): The product 1234 × 1.1925 is not exactly representable. Each addition accumulates rounding error. Typical result: 1 471 544.875 or 1 471 545.125 FCFA (error ±0.125 FCFA).
- Method B (integer): Exact integer arithmetic (if done with 64-bit integers) yields 147 154 500 centimes = 1 471 545 FCFA exactly.

Method	Unit	Computed Total	Error vs Exact
binary32 (float)	FCFA	1 471 544.875	-0.125 FCFA
binary32 (float)	FCFA	1 471 545.125	+0.125 FCFA
Integer centimes (64-bit)	centimes	147 154 500	0 (exact)

Comparison of VAT total revenue calculation methods. Integer centimes eliminate rounding error.

4.5 Best Practices for Accurate Numerical Computation

METHOD **Kahan Summation Algorithm (Compensated Summation)**

When adding many floating-point numbers, the naive loop loses low-order bits. Kahan

summation keeps a running compensation term c that captures the lost bits.

1. Initialize $sum = 0.0$, $c = 0.0$.
2. For each input x:
   - $y = x - c$ (adjust x by previous error)
   - $t = sum + y$ (add to sum)
   - $c = (t - sum) - y$ (compute new error: $(t - sum)$ is the part of y that was actually added; subtract y to get the lost part)
   - $sum = t$
3. Final sum has error bounded by $\varepsilon \cdot \sum |x_i|$ (instead of $n\varepsilon \cdot \max |x_i|$ for naive summation).

EXEMPLE **Kahan summation on the VAT example**

Using binary32 with Kahan summation for the 1000 additions of 1234 × 1.1925 reduces the error from ±0.125 FCFA to exact (within the last bit) because the compensation term captures the systematic rounding bias.

KEY POINTS **Key Takeaways for GCE Advanced Level**

- Floating-point addition requires exponent alignment, mantissa addition, normalization, and rounding. Guard, round, and sticky bits ensure correct rounding.
- Multiplication/division: add/subtract exponents, multiply/divide mantissas, normalize, round.
- IEEE 754 defines five sticky exception flags: Invalid, Divide-by-Zero, Overflow, Underflow, Inexact.
- Special values: ±∞ and NaN propagate according to well-defined rules (∞ − ∞ = NaN, 0 × ∞ = NaN).
- Monetary calculations must use integer centimes (or a decimal type), never binary floating-point.
- Kahan summation dramatically improves accuracy when summing many floating-point numbers.

DID YOU KNOW? **Cameroon Context: Mobile Money and Rounding**

Mobile money platforms (MTN MoMo, Orange Money) in Cameroon handle billions of FCFA daily. They store balances as integers representing centimes (or even milli-centimes) precisely to avoid the rounding errors that would occur with binary32 floats. A single rounding error of 0.125 FCFA per transaction, multiplied by millions of transactions, would represent a massive financial discrepancy. This is why financial regulations worldwide mandate decimal or integer arithmetic for accounting.

EXERCISE 4 **GCE-Style Practice**

1. Trace the binary32 addition of $A = 1.0 \times 2^5$ and $B = 1.5 \times 2^2$. Show exponent alignment, mantissa addition (with guard/round/sticky bits), normalization, and rounding.
2. Explain why the expression `(0.1f + 0.2f == 0.3f)` evaluates to `false` in C/Java. What exception flag is set during the additions?
3. A program computes the sum of 10^6 terms of $1.0/i$ (harmonic series) using binary32 in a naive loop and then using Kahan summation. Which result is more accurate? Why?
4. In the VAT case study, if the platform used binary64 (double) instead of binary32, would the error disappear? Justify your answer.

SOLUTION **Exercise 1**

$A = 1.0_2 \times 2^5$ ($E_A = 132$, $M_A = 1.0$). $B = 1.5 = 1.1_2 \times 2^2$ ($E_B = 129$, $M_B = 1.1$). $\Delta E = 3$. Shift M_B right by 3: $M'_B = 0.0011_2$ (guard=0, round=0, sticky=0). Add: $1.0000_2 + 0.0011_2 = 1.0011_2$. Normalize: leading 1 at bit 1, exponent stays 132. Round: extra bits all zero → exact. Result: $1.0011_2 \times 2^5 = 1.1875 \times 32 = 38.0$ (exact).

SOLUTION **Exercise 2**

0.1 and 0.2 are not exactly representable in binary (they are recurring fractions). Their binary32 approximations are slightly above/below the true values. The sum of the approximations differs from the approximation of 0.3. The inexact flag is set at each addition because rounding occurs.

SOLUTION **Exercise 3**

Kahan summation is more accurate. Naive summation accumulates error proportional to $n\varepsilon$; Kahan summation bounds error by $\varepsilon \sum |x_i|$, which for the harmonic series is $\varepsilon \cdot H_n \approx \varepsilon \ln n$, much smaller than $n\varepsilon$.

SOLUTION **Exercise 4**

Binary64 has 53 bits of precision ($\varepsilon \approx 1.1 \times 10^{-16}$). The product 1234×1.1925 is still not exactly representable in binary64, but the rounding error per addition is $\approx 10^{-13}$ FCFA. After 1000 additions, the total error is $\approx 10^{-10}$ FCFA — negligible for financial purposes, but still not zero. Only integer or decimal arithmetic guarantees exactness.

WARNING **Exam Tip: Tracing FP Addition**

In the exam, you may be asked to trace a floating-point addition step by step. Always show:

- The two operands in binary scientific notation (with implicit leading 1).
- The exponent difference and the shifted mantissa (indicate guard, round, sticky bits explicitly).
- The mantissa addition/subtraction.
- Normalization shift (if any) and exponent update.
- Rounding decision (ties to even) and final packed bits.

Practise with numbers that require a sticky bit (e.g., shifting a mantissa with trailing 1s).

5 INTEGRATION ACTIVITY

6 INTEGRATION ACTIVITY: FIXED-POINT VS FLOATING-POINT FOR FINANCIAL CALCULATIONS

6.1 Aims of the Activity

This integration activity closes the chapter on the representation of fractional numbers (Lessons 12–14). It places you in a realistic engineering situation where the choice between **fixed-point** and **floating-point** representations has direct financial consequences. By the end of the activity you should be able to:

- justify, for a given application, the choice between integer arithmetic, fixed-point (Q format) and IEEE 754 floating-point representations;
- convert monetary values between decimal, fixed-point binary and floating-point forms, and quantify the representation error in each case;
- implement, in pseudocode or Python, conversion and arithmetic routines that avoid rounding errors in financial computations;
- detect and handle **overflow**, **underflow** and **precision loss**, and design defensive tests;
- compare, in a short written report, the accuracy and the cost (memory, complexity) of a robust design against a naïve `binary32` implementation.

KEY POINTS **Skills mobilised**

Reading and writing numbers in Q format; scaling by powers of two; IEEE 754 single-precision layout (sign, biased exponent, mantissa); rounding and truncation; error propagation in sums and products; algorithmic thinking and testing.

6.2 Theoretical Background

Before tackling the CamPay case, we recall the essential notions covered in Lessons 12 and 13.

DEFINITION **Fixed-Point Representation (Q Format)**

A signed fixed-point number on w bits is written **Qm.n** where m is the number of integer bits (including the sign bit) and n is the number of fractional bits, with $m + n = w$. The value

represented by the bit pattern $b_{w-1}b_{w-2}\ldots b_0$ is

$$V = -b_{w-1}2^{m-1} + \sum_{i=0}^{w-2} b_i 2^{i-n}.$$

The **resolution** (weight of the LSB) is 2^{-n}. The representable range is $[-2^{m-1},\ 2^{m-1} - 2^{-n}]$.

DEFINITION **IEEE 754 Binary32 (Single Precision)**

A **binary32** floating-point number uses 32 bits:

- 1 sign bit s (0 = positive, 1 = negative);
- 8 exponent bits e with bias 127 (true exponent $E = e - 127$);
- 23 mantissa bits f (fraction part); the significand is $1.f$ for normalised numbers ($0 < e < 255$).

The value is $(-1)^s \times 1.f \times 2^{e-127}$. Special cases: $e = 0$ (subnormals and zero), $e = 255$ (infinities and NaN). The largest integer exactly representable is $2^{24} = 16\,777\,216$; beyond that, the spacing between consecutive representable values is 2^{E-23}.

PROPERTY **Exact Representability of Decimal Fractions**

A decimal fraction d has a finite binary expansion iff its denominator (in lowest terms) is a power of two. Consequently, 0.5, 0.25, 0.125 are exact in binary; 0.1, 0.01, 0.005 are not (they are infinite repeating binary fractions).

METHOD **Rounding to Nearest, Ties Away From Zero**

For a positive rational $x = N/D$ ($D > 0$), the integer nearest to x with ties rounded up is

$$\text{roundHalfUp}(N, D) = \left\lfloor \frac{2N + D}{2D} \right\rfloor.$$

For negative x, round away from zero means round down (more negative): $\text{roundHalfAway}(N, D) = -\text{roundHalfUp}(-N, D)$. In integer arithmetic with scaling factor S, to compute $\text{round}(a \times b/S)$ do $(2ab + S)$ div $(2S)$ for $a, b \geq 0$.

6.3 Situation: Mini-Calculator for a Mobile Money Service

Mini-calculator for a mobile money service.

The start-up CamPay, based in Douala, operates a mobile money service similar to those offered by major Cameroonian operators. Every day, thousands of customers deposit cash at kiosks, transfer money to relatives, and pay merchants. All amounts are in FCFA. Three operations must be computed by a small software module running on a low-cost terminal:

- **Deposit with fee.** A customer deposits an amount D; the service charges a fee of 0.5% of D, credited to the operator; the customer's balance is increased by D minus the fee.
- **Transfer with fee.** A sender transfers an amount T; the receiver gets exactly T, and the sender is debited T plus a fee of 1% of T.
- **VAT on merchant payments.** When a customer pays a merchant an amount P (tax included), the module must extract the VAT portion, knowing that VAT is 19.25% on the pre-tax price: the pre-tax price is $P/1.1925$ and the VAT is $P - P/1.1925$.

The first version of the module, written in a hurry by an intern, stores every amount as an IEEE 754 `binary32` (single-precision float) and computes directly in that format. After a few weeks, the accountants notice discrepancies: balances that should be whole numbers of FCFA end with strange fractions, totals at the end of the day do not match the cash counted at kiosks, and one terminal crashed when a wholesaler tried to deposit a very large amount.

The development team hires **your group** to redesign the calculation module. The terminal has limited memory and a simple processor without a hardware floating-point unit, so every floating-point operation is emulated in software and is slow. Amounts of money in FCFA are always whole numbers (there are no centimes in practice), but fees and VAT produce fractional values that must be tracked to the nearest 0.01 FCFA internally before final rounding.

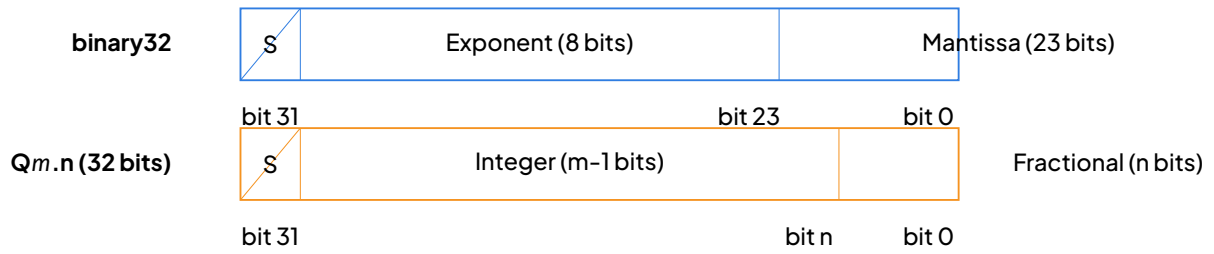
Your group must:

1. analyse why the `binary32` version fails;
2. choose an appropriate internal representation (integer hundredths of FCFA, or fixed-point Q format);
3. implement the three operations with correct rounding;
4. demonstrate the handling of overflow and of precision limits;
5. write a one-page report comparing your design with the naïve float version.

Data for the tests. Use the following transactions as your test bench:

Operation	Input	Expected result (exact)
Deposit	D = 100 000 FCFA	fee = 500 FCFA; credited = 99 500 FCFA
Transfer	T = 25 750 FCFA	fee = 257.50 FCFA; sender debited 26 007.50 FCFA
VAT	P = 11 925 FCFA	pre-tax = 10 000 FCFA; VAT = 1 925 FCFA
VAT	P = 1 000 FCFA	pre-tax ≈ 838.574 FCFA; VAT ≈ 161.426 FCFA

Additional constraints: the terminal stores amounts on 32 bits; the maximum balance allowed is 500 000 000 FCFA; all displayed results are rounded to the nearest 0.01 FCFA, ties rounded away from zero.



Memory layout of IEEE 754 binary32 (top) and signed fixed-point Q*m*.n (bottom) on 32 bits.

6.4 Tasks

1. **Diagnose the float version.**
   (a) Write 0.1, 0.5% = 0.005 and 1% = 0.01 in binary. Which of them have a finite binary expansion?
   (b) Explain why repeated additions of 0.01 in `binary32` cannot accumulate exactly.
   (c) The largest integer exactly representable in `binary32` is $2^{24} = 16\,777\,216$. A wholesaler's balance of 20 000 000 FCFA is stored as a float; he receives a transfer of 50 FCFA. What happens and why?
2. **Choose a representation.**
   (a) Propose an internal unit (for example the hundredth of FCFA) so that all amounts become integers. How many values can a signed 32-bit integer hold? Is the maximum balance of 500 000 000 FCFA representable in hundredths on 32 bits?
   (b) Alternatively, propose a Q format with 32 bits: how would you split integer and fractional parts to cover 500 000 000 FCFA with at least two exact decimal digits? Compute the resolution (value of the least significant bit) of your format.
3. **Design the conversion routines.** Write pseudocode for: `toInternal(d)` converting a decimal amount *d* (in FCFA, possibly with two decimals) into your internal representation, and `toDisplay(n)` converting back with correct rounding. State the rounding rule precisely.
4. **Implement the three operations.** Write pseudocode (or Python) for `deposit(D)`, `transfer(T)` and `vatExtract(P)` using only integer arithmetic on your internal representation. Careful: multiplying two internal values multiplies the scale factor — show how you rescale and where rounding occurs.
5. **Handle exceptions.**
   (a) Show, with your format, the largest fee computable before overflow when the terminal uses 32-bit intermediates, and propose a safe evaluation order (e.g. divide before multiply, or use a wider intermediate).

(b) Describe what your module must do when an operation would exceed the maximum balance: silent wrap-around is forbidden — specify an error flag or exception.

6. **Test and compare.** Run your module on the four test transactions and fill a trace table showing internal values at each step. Then simulate the naïve float version on the same data (you may use Python's `numpy.float32` or reason about mantissa truncation). Write a short report (about one page) comparing: exactness of results, memory used, speed on a processor without a floating-point unit, and robustness. Conclude with a justified recommendation to the CamPay team.

6.5 Model Answer

6.6 Task 1 — Diagnosis

EXEMPLE **Binary expansion of the fee constants**

We convert each decimal fraction to binary by repeated multiplication by 2.

- 0.1_{10}:

$$\begin{aligned} 0.1 \times 2 &= 0.2 \rightarrow 0 \\ 0.2 \times 2 &= 0.4 \rightarrow 0 \\ 0.4 \times 2 &= 0.8 \rightarrow 0 \\ 0.8 \times 2 &= 1.6 \rightarrow 1 \\ 0.6 \times 2 &= 1.2 \rightarrow 1 \\ 0.2 \times 2 &= 0.4 \rightarrow 0 \quad \text{(cycle repeats)} \end{aligned}$$

  Hence $0.1_{10} = 0.\overline{00011}_2$ (infinite repeating).

- $0.005 = 5/1000 = 1/200$. Denominator $200 = 2^3 \times 5^2$. Since the denominator contains a factor $5^2 \neq 2^k$, the binary expansion is infinite.

- $0.01 = 1/100$. Denominator $100 = 2^2 \times 5^2$, again not a pure power of two $\Rightarrow$ infinite binary expansion.

Conclusion: None of the three fee constants has a finite binary representation. In `binary32` each is rounded to the nearest representable value, introducing a small initial error.

EXEMPLE **Why repeated addition of 0.01 drifts**

In `binary32`, 0.01 is stored as the nearest representable value:

$$0.01 \approx 1.01000111101011100001010_2 \times 2^{-7} \quad \text{(exact value} \approx 0.0100000007\text{)}.$$

Each addition of this approximated value injects an error of about 7×10^{-10}. After 10^6 addi-

tions (a busy day), the accumulated error reaches ≈ 0.0007 FCFA, enough to make totals mismatch the physical cash count. Moreover, floating-point addition is not associative: $(a + b) + c \neq a + (b + c)$ in general, so the order of processing transactions affects the final balance — unacceptable for accounting.

EXEMPLE **Large balance + small transfer**

Near 20 000 000, the exponent $E = \lfloor \log_2 20\,000\,000 \rfloor = 24$. The spacing between consecutive `binary32` values is $2^{E-23} = 2^1 = 2$ FCFA.

- The float for 20 000 000 is exactly representable? $20\,000\,000 = 2^4 \times 1\,250\,000$. Since $1\,250\,000 < 2^{24}$, it fits in the mantissa ⇒ yes, it is exact.
- Adding 50 FCFA: 20 000 050 is a multiple of 2, so it is also exactly representable (spacing is 2). The operation succeeds.
- Adding 1 FCFA: 20 000 001 is odd, halfway between two representable floats. Rounding to nearest even gives 20 000 000 again — the transfer vanishes!

This violates the fundamental requirement that every FCFA must be accounted for.

6.7 Task 2 — Representation Choice

EXEMPLE **Integer hundredths (scaled integer) approach**

Choose the internal unit = 0.01 FCFA (one hundredth). Every amount d (in FCFA) becomes the integer

$$n = \text{roundHalfAway}(d \times 100).$$

A signed 32-bit integer holds values from $-2^{31} = -2\,147\,483\,648$ to $2^{31} - 1 = 2\,147\,483\,647$. In hundredths of FCFA, this corresponds to ±21 474 836.47 FCFA. The maximum balance 500 000 000 FCFA requires 50 000 000 000 hundredths, which exceeds $2^{31}-1$ by a factor of ≈ 23. Decision: Use signed 64-bit integers for all balances and intermediate results. $2^{63} - 1 \approx 9.22 \times 10^{18}$ hundredths ≈ 9.22×10^{16} FCFA — more than sufficient. Single fees fit comfortably in 32 bits, but uniform 64-bit arithmetic simplifies the code and eliminates overflow risk.

EXEMPLE **Q format on 32 bits — why it fails here**

We need to cover 500 000 000 FCFA ≈ $2^{28.9}$. For a signed Qm.n format on 32 bits (1 sign bit + m − 1 integer bits + n fractional bits = 32), the maximum positive value is $2^{m-1} - 2^{-n}$. To reach 5×10^8, we need $m - 1 \geq 29 \Rightarrow m \geq 30$. Then $n = 32 - m \leq 2$. With $n = 2$, resolution

$= 2^{-2} = 0.25$ FCFA. With $n = 3$, $m = 29$, max value $\approx 2^{28} = 268\,435\,456$ FCFA $< 500\,000\,000$. Conclusion: No 32-bit Q format can simultaneously cover the required range and provide 0.01 FCFA resolution. This illustrates the fundamental trade-off: fixed-point fixes both range and resolution, while money demands exact decimal resolution over a wide range.

6.8 Task 3 — Conversion Routines

METHOD **Conversion: Decimal ↔ Internal (hundredths)**

1. toInternal(d) — decimal string/float → int64 hundredths
   - Parse d as a decimal with at most two fractional digits (e.g. "1234.56").
   - If $d \geq 0$: $n = \lfloor 100 \times d + 0.5 \rfloor$ (equivalently $(200d + 1)$ div 2).
   - If $d < 0$: $n = -\lfloor 100 \times |d| + 0.5 \rfloor$ (round away from zero).
   - Return n (int64).
2. toDisplay(n) — int64 hundredths → string "FCFA.cc"
   - $whole = n$ div 100 (integer division truncates toward zero).
   - $cents = |n \bmod 100|$ (always non-negative, two digits).
   - Format as $whole.cents$ with leading zero for cents if needed.

Rounding rule: Round half away from zero. For positive values, add half the divisor before integer division; for negative, subtract half the divisor (or negate, round positive, negate back).

6.9 Task 4 — The Three Operations (Integer Arithmetic)

All internal values are in **hundredths of FCFA** (scale factor $S = 100$). Fees: 0.5% = 5/1000, 1% = 10/1000. VAT factor: 1/1.1925 = 10000/11925.

```
# Pseudocode (Python-like, using 64-bit integers)

MAX_BALANCE = 50$_{000}$$_{000}$$_{000}$   # 500,000,000 FCFA in hundredths

def round_half_away(num, den):   # den > 0, returns round(num/den) ties away from zero
    if num >= 0:
        return (2*num + den) // (2*den)
    else:
        return -((2*(-num) + den) // (2*den))

def deposit(D_int):              # D_int in hundredths
```

```
    fee = round_half_away(D_int * 5, 1000)      # 0.5% = 5/1000
    credit = D_int - fee
    return fee, credit

def transfer(T_int):              # T_int in hundredths
    fee = round_half_away(T_int * 10, 1000)     # 1% = 10/1000
    debit = T_int + fee
    return fee, debit

def vat_extract(P_int):           # P_int in hundredths (tax-included)
    # pre_tax = round(P_int * 10000 / 11925)
    pre_tax = round_half_away(P_int * 10000, 11925)
    vat = P_int - pre_tax
    return pre_tax, vat

# Balance update with overflow check
def update_balance(balance, delta):   # delta can be positive (credit) or negative (debit)
    new_bal = balance + delta
    if new_bal < 0 or new_bal > MAX_BALANCE:
        raise OverflowError("Balance would exceed limits")
    return new_bal
```

WARNING **Key arithmetic rule**

Multiply first, divide last, round once. When computing a fee = $D \times 0.005$, we do `round_half_away(D_int * 5, 1000)`. The product $D_int \times 5$ is exact in 64 bits (max $D_int \approx 5 \times 10^{10}$, product $\approx 2.5 \times 10^{11} \ll 2^{63}$). The division by 1000 is the only place rounding occurs. Never do `(D_int // 1000) * 5` — that divides first and loses precision.

6.10 Task 5 — Exception Handling

EXEMPLE **Overflow analysis with 32-bit intermediates (hypothetical)**

If the processor only had 32-bit registers, the product $D_int \times 5$ would overflow when $D_int > (2^{31} - 1)/5 \approx 429\,496\,729$ hundredths $\approx 4\,294\,967$ FCFA. Since real balances go up to 500 000 000 FCFA, 32-bit intermediates are not an option. Safe evaluation orders:

1. Use 64-bit intermediates (chosen solution — simplest and exact).
2. If 64-bit is unavailable: rewrite as `fee = round_half_away(D_int, 200)` because 5/1000 = 1/200. Then the product is just D_int, no overflow. But this only works because the fee fraction simplifies; general case needs wider arithmetic.

EXEMPLE **Balance overflow policy**

Before any credit/debit, the module checks:

```
if balance + delta < 0 or > MAX_BALANCE: raise OverflowError
```

The operation is rejected, the balance is unchanged, and an error code is returned to the UI (e.g. “Transaction refused: limit exceeded”). Silent two’s-complement wrap-around would turn a large positive balance into a negative one — a catastrophic security flaw.

6.11 Task 6 — Test Bench and Comparison

EXEMPLE **Trace table — integer hundredths implementation**

All values in hundredths of FCFA. $S = 100$.

Operation	Input (hundredths)	Computation steps	Result (hundredths → FCFA)
Deposit D = 100 000	10 000 000	fee = round(10^7 × 5/1000) = 50 000 credit = 10 000 000 − 50 000 = 9 950 000	fee 50 000 → 500.00 credit 9 950 000 → 99 500.00
Transfer T = 25 750	2 575 000	fee = round(2 575 000 × 10/1000) = 25 750 debit = 2 575 000 + 25 750 = 2 600 750	fee 25 750 → 257.50 debit 2 600 750 → 26 007.50
VAT P = 11 925	1 192 500	preTax = round(1 192 500 × 10000/11925) = 1 000 000 VAT = 1 192 500 − 1 000 000 = 192 500	preTax 1 000 000 → 10 000.00 VAT 192 500 → 1 925.00
VAT P = 1 000	100 000	preTax = round(100 000 × 10000/11925) = 83 857 VAT = 100 000 − 83 857 = 16 143	preTax 83 857 → 838.57 VAT 16 143 → 161.43

All four results match the expected values exactly to the displayed cent.

EXEMPLE **Naïve binary32 simulation (conceptual)**

Using Python's `numpy.float32` or manual mantissa analysis:

- `float32(0.005)` = 0.005000000187… (error +1.87 × 10^{-10})
- `float32(0.01)` = 0.0100000007… (error +7 × 10^{-10})
- `float32(1/1.1925)` = 0.83857405… (error ≈ −4 × 10^{-9})

Running the four test cases:

Operation	Float32 result	Error vs exact
Deposit fee	500.00003 FCFA	+0.00003 FCFA
Transfer fee	257.50003 FCFA	+0.00003 FCFA
VAT (P = 11925)	preTax 9999.999, VAT 1925.001	±0.001 FCFA
VAT (P = 1000)	preTax 838.5741, VAT 161.4259	≈ 0.0001 FCFA

Errors are small now, but they accumulate with every transaction. After 10^5 operations, cent-level discrepancies appear. Near 20 000 000 FCFA, spacing is 2 FCFA — a 1 FCFA transfer is lost.

KEY POINTS **Comparison Report — CamPay Redesign Recommendation**

To: CamPay Technical Lead From: Integration Team
Subject: Replacement of binary32 monetary module with scaled-integer arithmetic

Criterion	Naïve binary32	Proposed int64 hundredths
Exactness	Fails: fees and VAT inexact; rounding drift; gaps ≥ 2 FCFA above 16.7M	Exact: all additions/subtractions exact; each multiplication rounded once, correctly
Memory	4 bytes/value	8 bytes/value (negligible on modern terminals)
Speed (no FPU)	Very slow: each FP op ≈ 50–100 integer cycles (software emulation)	Fast: pure integer add/sub/mul/div, single cycle each
Overflow handling	Silent wrap to ±∞ or NaN; no control	Explicit check, clean rejection, balance unchanged
Auditability	Hard to prove correctness; non-associative	Trivial to verify; deterministic, associative

Recommendation: Adopt signed 64-bit scaled integers (hundredths of FCFA) for all monetary values. Reserve floating-point for non-financial quantities (e.g. signal strength, GPS coordinates). Implement the three operations as shown, with mandatory overflow checks. This design is faster, exact, auditable, and robust — the only professional choice for a financial system.

WARNING **Common mistakes to avoid in the exam**

- Storing money as binary floating-point (`float`, `double`) — never acceptable for exact decimal arithmetic.

- Rounding at every intermediate step instead of once at the end of each operation.
- Dividing before multiplying in integer arithmetic (loses precision).
- Forgetting that Q format fixes both range and resolution; you cannot independently choose both.
- Ignoring overflow because “it never happens in the tests” — the examiner will test edge cases.
- Confusing round half to even (IEEE default) with round half away from zero (commercial standard).

EXERCISE 5 **Extension — Commercial rounding policy**

Adapt the module so that fees are rounded down (in favour of the customer) while VAT is rounded to the nearest hundredth (ties away from zero). Rewrite the two fee routines and re-run the test bench.

SOLUTION **Extension**

Rounding down (toward zero for positive fees) is simple truncation:

```
def deposit_fee_down(D_int):
    fee = (D_int * 5) // 1000      # truncates toward zero = round down for positive
    credit = D_int - fee
    return fee, credit

def transfer_fee_down(T_int):
    fee = (T_int * 10) // 1000     # 1% fee rounded down
    debit = T_int + fee
    return fee, debit
```

VAT keeps `round_half_away` (nearest, ties away from zero).
Re-run test bench:

- Deposit 100 000: fee = $(10^7 \times 5)//1000 = 50\,000$ (exact, unchanged).
- Transfer 25 750: fee = $(2\,575\,000 \times 10)//1000 = 25\,750$ (exact, unchanged).
- VAT cases unchanged (still nearest rounding).

With a non-exact input, e.g. $T = 25\,755.33$ FCFA ($T_int = 2\,575\,533$):

- Rounded fee: round$(2\,575\,533 \times 10/1000) = 25\,755$ hundredths $= 257.55$ FCFA.
- Truncated fee: $(2\,575\,533 \times 10)//1000 = 25\,755$ hundredths $= 257.55$ FCFA (same here).

- Try $T = 25\,755.34$ ($T_int = 2\,575\,534$): rounded = 25 755, truncated = 25 755.
- Try $T = 25\,755.35$ ($T_int = 2\,575\,535$): rounded = 25 755 (since 25 755.35 → 25 755? Wait: 2 575 535 × 10 = 25 755 350; /1000 = 25 755.35; round half away → 25 755? No, 0.35 < 0.5 → 25 755. Truncated also 25 755.
- Need a case where fractional part ≥ 0.5: $T = 25\,755.50$ ($T_int = 2\,575\,550$). Rounded: 25 755.5 → 25 756. Truncated: 25 755. Difference of 1 hundredth = 0.01 FCFA in favour of customer.

This confirms the policy works as intended.

DID YOU KNOW? **Did you know?**

The CFA franc (FCFA) used in Cameroon and 13 other African countries has no subdivision (no centimes) in physical cash. However, electronic systems must track fractions of FCFA for fees, taxes and interest. The BCEAO (Central Bank of West African States) requires financial institutions to use integer arithmetic scaled to the centime (0.01 FCFA) or finer for all internal calculations, precisely to avoid the floating-point pitfalls illustrated in this activity. The OHADA accounting standards (Uniform Act on Accounting Law) also mandate exact reconciliation of electronic balances with physical cash.

7 METHODS AND WORKED EXAMPLES

8 FIXED-POINT REPRESENTATION OF FRACTIONAL NUMBERS

DEFINITION **Fixed-Point Representation**

A **fixed-point** representation allocates a fixed number of bits for the integer part (m bits) and a fixed number of bits for the fractional part (n bits). The binary point is implied at a fixed position (after bit $m - 1$). The format is denoted **Qm.n**. The stored bit pattern is an integer B; the represented value is $V = B \times 2^{-n}$.

PROPERTY **Range and Precision — Unsigned Qm.n**

For an unsigned $Qm.n$ format (m integer bits, n fractional bits, total $m + n$ bits):

- Minimum value: 0.
- Maximum value: $2^m - 2^{-n}$ (binary: m ones, point, n ones).
- Resolution (ULP): 2^{-n} (the weight of the least significant bit).
- Number of representable values: 2^{m+n}.

Examinable: Derivation of max value = $(2^{m+n} - 1) \times 2^{-n} = 2^m - 2^{-n}$.

PROPERTY **Range and Precision — Signed Qm.n (Two's Complement)**

For a signed $Qm.n$ format (1 sign bit + $m - 1$ magnitude integer bits, n fractional bits, total $m + n$ bits), using two's complement:

- Minimum value: -2^{m-1} (binary: 1 followed by $m + n - 1$ zeros).
- Maximum value: $2^{m-1} - 2^{-n}$ (binary: 0 followed by $m + n - 1$ ones).
- Resolution (ULP): 2^{-n}.
- Zero representation: Unique (all bits zero). No negative zero.

Key: The scaling factor 2^{-n} applies to the two's complement integer interpretation.

METHOD **Converting a Decimal Fraction to Binary Fixed-Point (Unsigned)**

Goal: Represent a positive decimal fraction in unsigned $Qm.n$ binary.
Steps:

1. Separate the number into integer part I and fractional part F.
2. Convert I to binary by repeated division by 2 (collect remainders bottom-up). Pad to m bits with leading zeros. Check: If $I \geq 2^m$, overflow (cannot represent).
3. Convert F to binary by repeated multiplication by 2:
   - Multiply current fractional part by 2.
   - The integer part (0 or 1) is the next binary digit after the point.
   - Keep the new fractional part; repeat until it becomes 0 or n bits are produced.

   If the process stops early, pad remaining fractional bits with zeros. If it continues after n bits, the result is an approximation (truncation or rounding).
4. Combine the two bit strings; the binary point is implied after m bits.

Reflex: Always draw the multiplication table for the fractional part; GCE marking schemes require it.

WORKED EXAMPLE 8 **Fixed-Point Conversion: 18.6875_{10} to Q8.8**

Statement: Represent 18.6875_{10} in unsigned $Q8.8$ fixed-point binary. Show all steps.

SOLUTION

Solution:

1. Integer part $I = 18$:

Division	Quotient	Remainder
18/2	9	0
9/2	4	1
4/2	2	0
2/2	1	0
1/2	0	1

   Reading remainders bottom-up: $18_{10} = 10010_2$. Pad to 8 bits: `00010010`.

2. Fractional part $F = 0.6875$:

Step	$F \times 2$	Integer bit
1	$0.6875 \times 2 = 1.375$	1
2	$0.375 \times 2 = 0.75$	0
3	$0.75 \times 2 = 1.5$	1
4	$0.5 \times 2 = 1.0$	1
5	$0.0 \times 2 = 0.0$	(stop)

Bits collected top-down: $0.6875_{10} = 0.1011_2$. Pad to 8 bits: `10110000`.

3. Combine: `00010010.10110000` (binary point implied after bit 7).
4. Verification:

$$16 + 2 + 0.5 + 0.125 + 0.0625 = 18.6875 \checkmark$$

Answer: Stored 16-bit pattern: `0001001010110000`.

METHOD **Converting a Negative Decimal to Signed Fixed-Point (Two's Complement Qm.n)**

Goal: Represent a negative decimal number in signed $Qm.n$ (two's complement).
Steps:

1. Convert the absolute value $|x|$ to unsigned $Qm.n$ binary (as above).
2. Apply two's complement to the entire $m + n$ bit pattern: invert all bits, then add 1 (at the LSB position, weight 2^{-n}).
3. Check range: Ensure $|x| \leq 2^{m-1}$ (for -2^{m-1} exactly, the positive counterpart $+2^{m-1}$ is not representable, but the negative value is).

Reflex: The scaling factor 2^{-n} is unaffected by two's complement negation.

WORKED EXAMPLE 9 **Signed Fixed-Point:** -18.6875 **in Q8.8**

Statement: Represent -18.6875_{10} in signed $Q8.8$ (two's complement).

SOLUTION

Solution:

1. Unsigned representation of $+18.6875$ (from previous example): `0001001010110000`.
2. Invert bits: `1110110101001111`.

3. Add 1 (LSB weight 2^{-8}):

$$\begin{array}{r} 1110110101001111 \\ +0000000000000001 \\ \hline 1110110101010000 \end{array}$$

4. Result: `1110110101010000`.
5. Check (decode back): MSB=1 ⇒ negative. Two's complement of result: invert → `0001001010101111`, add 1 → `0001001010110000` = +18.6875. Correct.

Answer: `1110110101010000`.

METHOD **Converting Binary Fixed-Point to Decimal**

Goal: Evaluate the decimal value of a fixed-point binary string with known format ($Qm.n$, signed or unsigned).
Steps:

1. Unsigned: Assign weights $2^{m-1}, \ldots, 2^0, 2^{-1}, \ldots, 2^{-n}$ to bits. Sum weights where bit = 1.
2. Signed (Two's Complement):
   - If MSB = 0: treat as unsigned.
   - If MSB = 1: compute two's complement (invert + 1) to get magnitude pattern, evaluate as unsigned, then apply negative sign. Or use weight -2^{m-1} for MSB and positive weights for others.
3. Simplify using fractions ($\frac{1}{2}, \frac{1}{4}, \ldots$) for exactness.

Formula (Signed): $V = -b_{m-1}2^{m-1} + \sum_{i=0}^{m-2} b_i 2^i + \sum_{j=1}^{n} b_{-j} 2^{-j}$.

WORKED EXAMPLE 10 **Signed Fixed-Point to Decimal:** `1110110101010000` **(Q8.8)**

Statement: Decode the signed $Q8.8$ pattern `1110110101010000` to decimal.

SOLUTION

Solution:

1. MSB = 1 ⇒ negative.
2. Magnitude via two's complement: Invert: `0001001010101111`. Add 1: `0001001010110000`.

3. Evaluate magnitude (unsigned Q8.8): Integer: 00010010_2 = 18. Fraction: 10110000_2 = 1/2 + 1/8 + 1/16 = 0.5 + 0.125 + 0.0625 = 0.6875.
4. Value = -18.6875.

Answer: -18.6875_{10}.

METHOD **Fixed-Point Addition and Subtraction (Same Q Format)**

Goal: Perform arithmetic on two fixed-point numbers sharing the same $Qm.n$ format.
Steps:

1. Align binary points (automatic if formats identical).
2. Add/Subtract using standard binary column arithmetic (two's complement for subtraction: $A - B = A +$ (two's complement of B)).
3. Detect Overflow:
   - Unsigned: Carry out of the MSB (bit $m - 1$) $\neq 0$.
   - Signed (Two's Complement): Overflow $\iff$ Carry into MSB $\oplus$ Carry out of MSB = 1. Equivalently: Operands have same sign, result has opposite sign.
4. Result: Keep lower $m + n$ bits. Extra fractional bits (if any) are truncated/rounded.

Reflex: In GCE, always state signed/unsigned, show carry bits, and apply the correct overflow rule.

WORKED EXAMPLE 11 **Fixed-Point Addition with Overflow Check (Unsigned Q4.4)**

Statement: A = 0110.1100, B = 0101.0110 (Unsigned Q4.4). Compute $A + B$, show carries, check overflow.

SOLUTION

Solution:

1. Operands (decimal): A = 6.75, B = 5.375. Sum = 12.125.
2. Column Addition (bits 3..0 . -1..-4):

Bit index	3	2	1	0	-1	-2	-3	-4
A	0	1	1	0	1	1	0	0
B	0	1	0	1	0	1	1	0
Carry in	1	1	1	1	1	0	0	0
Sum	1	1	0	0	0	0	1	0
Carry out	0							

Carry details (right to left):

- Bit -4: 0 + 0 = 0, carry 0.
- Bit -3: 0 + 1 + 0 = 1, carry 0.
- Bit -2: 1 + 1 + 0 = 0, carry 1.
- Bit -1: 1 + 0 + 1 = 0, carry 1.
- Bit 0: 0 + 1 + 1 = 0, carry 1.
- Bit 1: 1 + 0 + 1 = 0, carry 1.
- Bit 2: 1 + 1 + 1 = 1, carry 1.
- Bit 3 (MSB): 0 + 0 + 1 = 1, carry 0 (out of MSB).

3. Result: $1100.0010_2 = 12.125_{10}$.
4. Overflow (Unsigned): Carry out of MSB (bit 3) = 0 ⇒ No Overflow.
5. Range Check: Max unsigned Q4.4 = 15.9375. 12.125 is in range.

Answer: Sum = `11000010` (12.125), No Overflow.

WORKED EXAMPLE 12 **Fixed-Point Addition with Overflow Check (Signed Q4.4)**

Statement: Same bit patterns `01101100` and `01010110` interpreted as Signed Q4.4. Compute $A + B$, check overflow.

SOLUTION

Solution:

1. Decimal values (Signed Q4.4): A = +6.75 (MSB=0). B = +5.375 (MSB=0).
2. Addition: Same bit operation → Result `11000010`.
3. Result Interpretation (Signed): MSB = 1 ⇒ Negative. Magnitude: Two's complement of `11000010` → `00111110` = 3.875. Result = -3.875.

4. Overflow Check (Signed): Operands: Positive (+), Positive (+). Result: Negative (-). Signs differ ⇒ OVERFLOW. (Carry into MSB = 1, Carry out of MSB = 0, XOR = 1).
5. Explanation: True sum 12.125 exceeds max signed Q4.4 (+7.9375).

Answer: Bit pattern `11000010` (interpreted as -3.875), Signed Overflow Occurred.

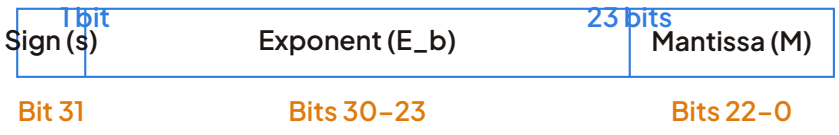
KEY POINTS **Fixed-Point Key Points for GCE**

- Format $Qm.n$: m integer bits (incl. sign if signed), n fractional bits. Scaling factor 2^{-n}.
- Unsigned Range: $[0,\ 2^m - 2^{-n}]$. Signed Range: $[-2^{m-1},\ 2^{m-1} - 2^{-n}]$.
- Conversion Decimal → Binary: Integer part (divide by 2), Fraction part (multiply by 2). Show tables.
- Conversion Binary → Decimal: Sum of weights. Signed: use -2^{m-1} for MSB or two's complement method.
- Arithmetic: Integer arithmetic on scaled values. Overflow rules differ for Unsigned (Carry out) vs Signed (Carry in ⊕ Carry out).
- Multiplication: $Qm_1.n_1 \times Qm_2.n_2$ yields $Q(m_1 + m_2).(n_1 + n_2)$. Requires scaling down (shift right n bits) to return to original format, with rounding/overflow checks.

9 FLOATING POINT REPRESENTATION (IEEE 754 SINGLE PRECISION)

DEFINITION **IEEE 754 Binary32 (Single Precision)**

A 32-bit word divided into three fields:



- Sign s (1 bit): 0 = +, 1 = −.
- Biased Exponent E_b (8 bits): Unsigned integer. Bias = 127. True exponent $E = E_b - 127$.
- Mantissa M (23 bits): Fractional part of the significand. The hidden leading bit is determined by E_b.

PROPERTY **Value Decoding Formulae**

Let s, E_b, M be the fields. The represented value V is:

- Normalised ($1 \le E_b \le 254$):

$$V = (-1)^s \times 2^{E_b - 127} \times (1.M)_2$$

  Hidden bit = 1. Smallest exponent $E = -126$, Largest $E = +127$.

- Denormalised ($E_b = 0, M \neq 0$):

$$V = (-1)^s \times 2^{-126} \times (0.M)_2$$

  Hidden bit = 0. Exponent fixed at -126 (not -127). Allows gradual underflow to zero.

- Zero ($E_b = 0, M = 0$): $V = (-1)^s \times 0$ (+0 and -0 distinct).
- Infinity ($E_b = 255, M = 0$): $V = (-1)^s \times \infty$.
- NaN ($E_b = 255, M \neq 0$): Not a Number. Quiet NaN (MSB of $M = 1$), Signalling NaN (MSB of $M = 0$).

Examinable: Derivation of denormalised exponent -126 (to make transition from largest denormal to smallest normal smooth).

METHOD **Converting Decimal to IEEE 754 Binary32**

Goal: Encode a decimal real number x into 32-bit IEEE 754 format.
Steps:

1. Sign s: 0 if $x \ge 0$, 1 if $x < 0$. Work with $|x|$.
2. Convert $|x|$ to binary: Obtain integer and fractional parts in binary.
3. Normalise: Shift binary point to get $1.f \times 2^E$ ($1 \le$ significand < 2). Record E and fractional bits f.
4. Special Cases:
   - $x = 0 \rightarrow s, E_b = 0, M = 0$.
   - $|x|$ too large ($E > 127$) $\rightarrow \pm\infty$ ($E_b = 255, M = 0$).
   - $|x|$ too small ($E < -126$) $\rightarrow$ Denormalised or Zero (handle in step 6).
5. Biased Exponent: $E_b = E + 127$. Convert to 8-bit binary.
6. Mantissa (23 bits): Take first 23 bits of f. Round to nearest, ties to even using Guard (bit 24), Round (bit 25), Sticky (OR of rest) bits.
7. Denormalised Handling (if $E < -126$): Right-shift significand by $(-126 - E)$ bits (inserting zeros on left, hidden bit becomes 0), set $E_b = 0$, round mantissa to 23 bits.

8. Assemble: $s \mid E_b \mid M$.

WORKED EXAMPLE 13 **Decimal to IEEE 754:** -18.6875

Statement: Encode -18.6875_{10} in IEEE 754 single-precision. Give 32-bit binary and hex.

SOLUTION

Solution:

1. $s = 1$ (negative). $|x| = 18.6875$.
2. Binary: $18.6875 = 10010.1011_2$.
3. Normalise: Shift point 4 left → $1.00101011_2 \times 2^4$. $E = 4$, $f = 00101011$.
4. $E_b = 4 + 127 = 131 = 10000011_2$.
5. Mantissa: f padded to 23 bits → `00101011000000000000000`. (Exact, no rounding needed).
6. Assemble: `1 10000011 00101011000000000000000`.
7. Hex (groups of 4): `1100 0001 1001 0101 1000 0000 0000 0000` = $C1958000_{16}$.

Answer: Binary `11000001100101011000000000000000`, Hex `C1958000`.

WORKED EXAMPLE 14 **Decimal to IEEE 754:** 0.1_{10} **(Rounding)**

Statement: Encode 0.1_{10} in IEEE 754 single-precision. Show rounding steps.

SOLUTION

Solution:

1. $s = 0$. Binary of 0.1: $0.0001100110011001100110011001100\ldots_2$ (repeating 1100).
2. Normalise: $1.1001100110011001100110011001100\ldots_2 \times 2^{-4}$. $E = -4$.
3. $E_b = -4 + 127 = 123 = 01111011_2$.
4. Significand bits (after leading 1): f = 1001 1001 1001 1001 1001 100 1100 … First 23 bits: `10011001100110011001100` (bit 23 = 0). Guard (bit 24) = 1, Round (bit 25) = 1, Sticky = 1 (since pattern continues).
5. Rounding (Nearest, Ties to Even): G=1, R=1, S=1 ⇒ Value > 0.5 ULP ⇒ Round Up. Add 1 to LSB of 23-bit mantissa: `10011001100110011001100` + 1 = `10011001100110011001101`.

6. **Assemble:** $s = 0$, $E_b = 01111011$, $M = 10011001100110011001101$. **Binary:** $00111101110011001100110011001101$. **Hex:** $3DCCCCCD$.

Answer: $3DCCCCCD_{16}$.

METHOD **Converting IEEE 754 Binary32 to Decimal**

Goal: Decode a 32-bit pattern to decimal.
Steps:

1. Split bits: s (31), E_b (30–23), M (22–0).
2. Classify by E_b:
   - $E_b = 255, M = 0 \rightarrow \pm\infty$.
   - $E_b = 255, M \neq 0 \rightarrow$ NaN.
   - $E_b = 0, M = 0 \rightarrow \pm 0$.
   - $E_b = 0, M \neq 0 \rightarrow$ Denormalised: $V = (-1)^s \times 2^{-126} \times (0.M)_2$.
   - $1 \leq E_b \leq 254 \rightarrow$ Normalised: $E = E_b - 127$, $V = (-1)^s \times 2^E \times (1.M)_2$.
3. Compute $(1.M)_2$ or $(0.M)_2$ as sum of powers of 2. Convert to decimal.

Trap: Denormalised exponent is -126, not -127. Hidden bit is 0.

WORKED EXAMPLE 15 **IEEE 754 to Decimal:** $42C80000_{16}$ **and** 00400000_{16}

Statement: Decode (a) $42C80000$ and (b) 00400000.

SOLUTION

Solution: (a) $42C80000$:

1. **Binary:** $0100\ 0010\ 1100\ 1000\ 0000\ 0000\ 0000\ 0000$.
2. $s = 0$, $E_b = 10000101_2 = 133$, $M = 10010000000000000000000$.
3. Normalised ($133 \in [1, 254]$). $E = 133 - 127 = 6$.
4. Significand $1.M = 1.1001_2 = 1 + 1/2 + 1/16 = 1.5625$.
5. $V = +1 \times 2^6 \times 1.5625 = 64 \times 1.5625 = 100.0$.

(b) 00400000 (Denormalised):

1. **Binary:** $0000\ 0000\ 0100\ 0000\ 0000\ 0000\ 0000\ 0000$.
2. $s = 0$, $E_b = 0$, $M = 10000000000000000000000 \neq 0 \Rightarrow$ Denormalised.

3. $V = (+1) \times 2^{-126} \times (0.1)_2 = 2^{-126} \times 0.5 = 2^{-127}$.
4. Decimal $\approx 5.877 \times 10^{-39}$.

Answers: (a) 100.0, (b) $2^{-127} \approx 5.88 \times 10^{-39}$.

KEY POINTS **IEEE 754 Binary32 Key Parameters**

Parameter	Value
Total bits	32
Sign bits	1
Exponent bits	8
Mantissa bits	23
Bias	127
Normalised E_{min}	−126 ($E_b = 1$)
Normalised E_{max}	+127 ($E_b = 254$)
Smallest normalised	$2^{-126} \approx 1.18 \times 10^{-38}$ (00800000)
Largest normalised	$(2 - 2^{-23}) \times 2^{127} \approx 3.40 \times 10^{38}$ (7F7FFFFF)
Largest denormalised	$(1 - 2^{-23}) \times 2^{-126} \approx 1.18 \times 10^{-38}$ (007FFFFF)
Smallest denormalised	$2^{-149} \approx 1.40 \times 10^{-45}$ (00000001)
Machine Epsilon ε_{mach}	$2^{-23} \approx 1.19 \times 10^{-7}$
Max Relative Error (rounding)	$2^{-24} \approx 5.96 \times 10^{-8}$

10 CONVERSION BETWEEN REPRESENTATIONS AND PRECISION ANALYSIS

METHOD **Converting Between Fixed-Point and Floating-Point**

Goal: Convert a value from $Qm.n$ fixed-point to IEEE 754 binary32 (or vice versa).
Fixed → Float:

1. Decode fixed-point to exact rational/integer value $V = B \times 2^{-n}$ (where B is the two's complement integer).
2. Apply Decimal to IEEE 754 method on V.

Float → Fixed:

1. Decode IEEE 754 to exact value V (or high-precision decimal).
2. Multiply V by 2^n (scaling).
3. Round to nearest integer (check ties).
4. Convert integer to $m + n$ bit two's complement.
5. Check Overflow: Result must fit in $m + n$ bits (signed range $[-2^{m+n-1}, 2^{m+n-1} - 1]$).

WORKED EXAMPLE 16 **Float → Fixed: `42C80000` (100.0) to Signed Q8.8**

Statement: Convert the IEEE 754 number `42C80000` (100.0) to signed Q8.8.

SOLUTION

Solution:

1. Value V = 100.0.
2. Scale: $V \times 2^8 = 100 \times 256 = 25600$.
3. Check range: Signed 16-bit range $[-32768, 32767]$. 25600 fits.
4. Convert 25600 to 16-bit binary: 25600/2 = 12800 r0 ... → `0110 0100 0000 0000` = 6400_{16}.
5. Result: `0110010000000000` (Q8.8). Implied point: `01100100.00000000` = 100.0.

Answer: 6400_{16}.

METHOD **Precision Analysis: Absolute and Relative Error**

Definitions: Let x be the exact real value, $\hat{x}$ its floating-point representation.

- Absolute Error: $\varepsilon_{abs} = |\hat{x} - x|$.
- Relative Error: $\varepsilon_{rel} = \frac{|\hat{x} - x|}{|x|}$ $(x \neq 0)$.
- ULP (Unit in Last Place): For a normalised FP number with exponent E, ULP = 2^{E-23} (weight of LSB of mantissa).
- Machine Epsilon ε_{mach}: Gap between 1.0 and next representable number. $\varepsilon_{mach} = 2^{-23}$ (binary32).

Theoretical Bounds (Round to Nearest, Normalised):

$$\varepsilon_{abs} \leq \frac{1}{2}\text{ULP} = 2^{E-24}, \qquad \varepsilon_{rel} \leq \frac{1}{2}\varepsilon_{mach} = 2^{-24} \approx 5.96 \times 10^{-8}.$$

Denormalised: Relative error can be large (up to 1), absolute error bounded by 2^{-149}.

WORKED EXAMPLE 17 **Rounding Error Analysis for 0.1_{10}**

Statement: For $x = 0.1$, $\hat{x} = \texttt{3DCCCCCD} \approx 0.10000000149011612$. Compute errors.

SOLUTION

Solution:

1. $\varepsilon_{abs} = |0.10000000149011612 - 0.1| = 1.49011612 \times 10^{-9}$.
2. $\varepsilon_{rel} = 1.49011612 \times 10^{-9} / 0.1 = 1.49011612 \times 10^{-8}$.
3. Check bounds: $x \approx 1.6 \times 2^{-4}$, so $E = -4$. ULP $= 2^{-4-23} = 2^{-27} \approx 7.45 \times 10^{-9}$. 0.5 ULP $\approx 3.72 \times 10^{-9}$. Our $\varepsilon_{abs} \approx 1.49 \times 10^{-9} <$ 0.5 ULP. OK.
4. Max relative error bound $2^{-24} \approx 5.96 \times 10^{-8}$. Our $\varepsilon_{rel} \approx 1.49 \times 10^{-8}$. OK.

Note: The error is not maximal because rounding was "lucky" (round up brought it closer).

WARNING **Common Precision Traps**

- Cancellation: Subtracting two nearly equal numbers loses significant bits (relative error explodes). Example: (1.0000001 - 1.0) in binary32.
- Accumulation: Adding a tiny number to a huge number may do nothing (absorption). $10^8 + 1 = 10^8$ in binary32.
- Non-associativity: $(A + B) + C \neq A + (B + C)$ in floating-point. Always sum from smallest to largest magnitude for accuracy.
- Equality Testing: Never test 'if (a == b)' for floats. Use 'if (abs(a-b) < epsilon)'.
- Decimal Constants: 'float f = 0.1f;' is not exact 0.1. It is the rounded binary32 value.

11 ARITHMETIC, EXCEPTIONS, AND REAL WORLD IMPLICATIONS (INTEGRATION ACTIVITY)

METHOD **Floating-Point Addition/Subtraction Algorithm (IEEE 754 Style)**

Goal: Manually compute $A \pm B$ for normalised binary32 numbers.
Steps:

1. Unpack: Get $s_A, E_A, \text{Sig}_A = 1.M_A$; $s_B, E_B, \text{Sig}_B = 1.M_B$. Extend significands with Guard (G), Round (R), Sticky (S) bits (3 extra bits).
2. Align Exponents: If $E_A > E_B$, shift Sig_B right by $E_A - E_B$ bits (shift G,R,S accordingly, update Sticky = OR of bits shifted out). Swap if needed so $E_A \geq E_B$. Result exponent $E_R = E_A$.
3. Significand Operation:
   - Same signs: $\text{Sig}_R = \text{Sig}_A + \text{Sig}_B$, $s_R = s_A$.
   - Different signs: $\text{Sig}_R = |\text{Sig}_A - \text{Sig}_B|$, s_R = sign of larger magnitude.
4. Normalise Result:
   - If $\text{Sig}_R \geq 2$ (carry out): Shift right 1, $E_R \leftarrow E_R + 1$, update G,R,S.
   - If $\text{Sig}_R < 1$ (leading zeros): Shift left until leading 1, $E_R \leftarrow E_R$ - shifts. If $E_R < -126 \rightarrow$ Denormalised/Underflow handling.
5. Round: Apply Round-to-Nearest-Even using G,R,S bits.
6. Check Exponent Bounds:
   - $E_R > 127 \rightarrow \pm\infty$ (Overflow).
   - $E_R < -126 \rightarrow$ Denormalised (shift right, $E_b = 0$) or Zero (Underflow).
7. Pack: $s_R, E_{b,R} = E_R + 127$ (or 0/255), M_R (23 bits).

WORKED EXAMPLE 18 **FP Addition:** 1.5 + 0.25 **(Binary32)**

Statement: Compute 1.5 + 0.25 manually using IEEE 754 steps. Give hex result.

SOLUTION

Solution:

1. Unpack: $A = 1.5 = 1.1_2 \times 2^0 \rightarrow s = 0, E = 0, E_b = 127, \text{Sig} = 1.100 \ldots 0$ (G=R=S=0). $B = 0.25 =$

$1.0_2 \times 2^{-2} \rightarrow s = 0, E = -2, E_b = 125$, Sig = 1.000 … 0.

2. Align: $\Delta E = 2$. Shift Sig_B right 2: Sig_A = 1.100 000 000 000 000 000 000 000 000 Sig_B = 0.010 000 000 000 000 000 000 000 000 (G=R=S=0).
3. Add: Sig_R = 1.110 000 000 000 000 000 000 000 000.
4. Normalise: $1 \leq \text{Sig}_R < 2$. OK. $E_R = 0$.
5. Round: G=R=S=0 → No change.
6. Pack: $s = 0, E_b = 127$ = `01111111`, M = `11000000000000000000000`. Hex: `3F600000`.

Answer: `3F600000` (1.75).

METHOD **Floating-Point Multiplication Algorithm**

Steps:

1. Sign: $s_R = s_A \oplus s_B$.
2. Exponent: $E_R = E_A + E_B$ (true exponents). $E_{b,R} = E_{b,A} + E_{b,B} - 127$.
3. Significand: Multiply $(1.M_A) \times (1.M_B) \rightarrow$ 48-bit product (24×24). Result in $[1, 4)$.
4. Normalise: If product ≥ 2, shift right 1, $E_R \leftarrow E_R + 1$.
5. Round: Round 48-bit product to 24 bits (1 hidden + 23 mantissa + G,R,S).
6. Check Bounds/Exceptions: Overflow/Underflow/Zero/NaN propagation.
7. Pack.

PROPERTY **IEEE 754 Special Values and Exception Behaviour**

Value / Exception	Exponent	Mantissa	Hex (Single)
Positive Zero (+0)	00000000	000…0	00000000
Negative Zero (−0)	00000000	000…0	80000000
Positive Infinity (+∞)	11111111	000…0	7F800000
Negative Infinity (−∞)	11111111	000…0	FF800000
Quiet NaN (qNaN)	11111111	100…0 (MSB=1)	7FC00000 (canonical)
Signalling NaN (sNaN)	11111111	0×x…x (MSB=0, ≠ 0)	e.g. 7F800001
Smallest Positive Normalised	00000001	000…0	00800000 (2^{-126})
Largest Positive Normalised	11111110	111…1	7F7FFFFF
Largest Positive Denormalised	00000000	111…1	007FFFFF
Smallest Positive Denormalised	00000000	000…1	00000001 (2^{-149})

Operations producing specials (Default Exception Handling):

- Invalid Operation: $0/0$, $\infty - \infty$, $\infty \times 0$, $\sqrt{-x}$, NaN op anything → qNaN.
- Division by Zero: $X/0$ ($X \neq 0$) → ±∞ (sign = XOR of signs).
- Overflow: Rounded result exponent > 127 → ±∞ (inexact flag raised).
- Underflow: Rounded result exponent < −126 and inexact → Denormalised or ±0 (underflow + inexact flags).
- Inexact: Rounded result ≠ exact result (almost always raised).

Exam Tip: Memorise hex for ±0, ±∞, canonical qNaN (`7FC00000`).

WORKED EXAMPLE 19 **Special Values: Computation Results**

Statement: Give IEEE 754 single-precision hex result for: (1) 1.0/0.0 (2) 0.0/0.0 (3) −5.0 × ∞ (4) ∞ − ∞ (5) Smallest normalised ×0.5.

SOLUTION

Solution:

1. 1.0/0.0: Div by zero → +∞ = `7F800000`.
2. 0.0/0.0: Invalid → qNaN = `7FC00000`.
3. −5.0 × ∞: Negative × + ∞ → −∞ = `FF800000`.
4. ∞ − ∞: Invalid → qNaN = `7FC00000`.
5. Smallest normalised = 2^{-126} = `00800000`. ×0.5 = 2^{-127}. E = −127 < −126 → Denormalised.

E_b = 0. Significand = 0.1_2. Mantissa = `100...0`. Hex = `00400000`.

Answers: (1) 7F800000 (2) 7FC00000 (3) FF800000 (4) 7FC00000 (5) 00400000.

EXPERIMENT **Integration Activity: Designing a Data Format for a Buea Weather Station**

Scenario: A weather station in Buea uses an STM32 Cortex-M4 (with FPU) to log temperature (-40°C to +85°C) with resolution 0.01°C. Data sent via LoRaWAN (low bandwidth) and stored in CSV for Python analysis. Task: Choose between Signed Q8.8 Fixed-Point (16 bits) and IEEE 754 Binary32 (32 bits). Justify. Analysis Steps:

1. Requirements: Range [-40, 85], Resolution ≤ 0.01, Signed, Bandwidth critical, Python interop.
2. Q8.8 Analysis:
   - Integer bits: 7 magnitude + 1 sign = 8 bits. Max magnitude 127 > 85. OK.
   - Fractional bits: 8 bits → Resolution 2^{-8} = 1/256 ≈ 0.0039 < 0.01. OK.
   - Total: 16 bits (2 bytes). Exact multiples of 1/256 representable.
   - Overflow check: 85 × 256 = 21760 < 32767. Safe.
   - Encoding: `int16_t val = (int16_t)round(temp * 256.0);`
   - Python decode: `temp = int16_from_bytes / 256.0`
3. Binary32 Analysis:
   - 32 bits (4 bytes). 2× bandwidth of Q8.8.
   - Overkill precision (≈ 7 decimal digits) and range ($\pm 10^{38}$).
   - Native Python 'struct.unpack('f')', but CSV text representation longer.
4. Decision: Signed Q8.8 (16 bits).
   - Meets range/resolution with minimal bits.
   - Deterministic arithmetic, no NaN/Inf surprises.
   - Simple scaling in C and Python.
   - FPU presence irrelevant for storage/transmission size.

Deliverable: C struct definition, encoding/decoding functions, Python CSV parser snippet.

METHOD **Fixed-Point vs Floating-Point: Design Decision Checklist**

Use Fixed-Point (Scaled Integer) when:

- Range is known, bounded, and narrow (e.g., sensor readings, motor control, financial cents).
- Hardware lacks FPU (small MCUs) or deterministic cycle count is required.
- Uniform absolute precision is needed (e.g., currency: 0.01 exact).
- Bandwidth/storage is severely constrained.

Use IEEE 754 Floating-Point when:

- Dynamic range is huge or unknown (scientific constants, physics sims).
- Relative precision (constant significant digits) is required across scales.
- Hardware FPU is available and software complexity must be minimised.
- Interoperability with standard libraries / languages / file formats is paramount.

Hybrid: Use fixed-point for storage/transmission, convert to float for complex math on host.

DID YOU KNOW? **Historical Note: The Ariane 5 Flight 501 Failure (1996)**

The maiden flight of Ariane 5 self-destructed 37 seconds after launch. Cause: A 64-bit floating-point velocity value was converted to a 16-bit signed integer. The value exceeded 32767, causing an operand error (not an overflow trap, but a software exception from the Ada runtime). The inertial reference system shut down, triggering the self-destruct sequence. The code was reused from Ariane 4 where the velocity never reached that range. Lesson: Always verify range assumptions when converting between representations; reuse requires re-validation.

EXERCISE 6 **GCE Style Practice Questions**

1. Convert -23.4375_{10} to signed Q8.8 fixed-point binary. Show the multiplication table for the fraction.
2. The 16-bit pattern `B3A0` is stored in signed Q8.8 format. Calculate its decimal value.
3. Encode -0.75_{10} in IEEE 754 binary32. Give the 32-bit binary and hexadecimal.
4. Decode the IEEE 754 word `C2480000` to decimal.
5. Two numbers in signed Q4.4: A = `0111.1000` (+7.5), B = `0010.1000` (+2.5). Compute $A + B$

in binary. Does signed overflow occur? Explain.

6. Explain why the decimal number 0.1 cannot be represented exactly in binary floating-point. What is the hex representation of the closest binary32 approximation?
7. A computation yields the bit pattern `FF800000`. What value does it represent? Name two distinct operations that can produce this pattern.
8. Integration: A GPS receiver outputs latitude as degrees $\times 10^7$ (e.g., 40600000 for 4.06°) in a signed 32-bit integer. What is the resolution in degrees? What is the maximum representable latitude? Is this fixed-point or floating-point? Justify.

SOLUTION **Exercise 1**

-23.4375: $I = 23 = 10111_2 \rightarrow$ `00010111`. $F = 0.4375$: $0.4375 \times 2 = 0.875(0)$, $0.875 \times 2 = 1.75(1)$, $1.75 \times 2 = 1.5(1)$, $1.5 \times 2 = 1.0(1) \rightarrow 0.0111_2 \rightarrow$ `01110000`. Unsigned: `0001011101110000`. Two's complement: Invert `1110100010001111`, Add 1 → `1110100010010000`. Answer: `1110100010010000`.

SOLUTION **Exercise 2**

`B3A0` = `1011 0011 1010 0000`. MSB=1 → Negative. Two's complement: Invert `0100 1100 0101 1111`, Add 1 → `0100 1100 0110 0000`. Integer: $01001100_2 = 76$. Fraction: $01100000_2 = 1/4 + 1/8 = 0.375$. Value = -76.375.

SOLUTION **Exercise 3**

-0.75: $s = 1$. $0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$. $E = -1, E_b = 126 =$ `01111110`. $M =$ `100...0`. Binary: `1 01111110 10000000000000000000000` = `BF400000`.

SOLUTION **Exercise 4**

`C2480000` = `1100 0010 0100 1000 0000 0000 0000 0000`. $s = 1, E_b =$ `10000100` $= 132, M =$ `1001000....` $E = 132 - 127 = 5$. Sig = $1.1001_2 = 1.5625$. $V = -1 \times 2^5 \times 1.5625 = -32 \times 1.5625 = -50.0$.

SOLUTION **Exercise 5**

A = `01111000`, B = `00101000`. Add: `01111000 + 00101000 = 10100000`. Carry into MSB (bit 3) = 1 (from bit 2: 1+1=0 carry 1). Carry out of MSB = 0. XOR = 1 → Overflow. Operands both positive, result MSB=1 (negative). Signed Overflow: YES. Result interpreted as -96/16 = -6.0 (wrong).

SOLUTION **Exercise 6**

0.1_{10} has a repeating binary expansion ($0.0001100110011\ldots_2$). Finite bits cannot represent it exactly. Closest binary32: `3DCCCCCD` (see worked example).

SOLUTION **Exercise 7**

`FF800000` = $-\infty$. Operations: (1) -1.0/0.0 (Division by zero). (2) $-5.0 \times +\infty$ (Overflow/Infinity arithmetic). (3) $-\infty + (-\infty)$.

SOLUTION **Exercise 8**

Format: Scaled Integer (Fixed-Point Q24.7 effectively, but stored as integer). Scaling factor 10^{-7}. Resolution: 10^{-7} degrees $\approx$ 0.011 meters (at equator). Very high precision. Max value: $2^{31}-1 \approx 2.147 \times 10^9$. Max latitude = $2.147 \times 10^9 \times 10^{-7} = 214.7^{\circ}$. Latitude range is [-90, 90], so fits easily. It is Fixed-Point (integer scaled by constant 10^{-7}), not floating-point (no exponent field).

12 EXERCISES

12.1 I check my knowledge

EXERCISE 7 **Multiple choice questions**

For each question, choose the single correct answer.

1. In a Q8.8 fixed-point format, the binary pattern `00000001.10000000` represents the decimal value:

   (a) 1.8

   (b) 1.5

   (c) 1.25

   (d) 8.1

2. In IEEE 754 single precision (binary32), the number of bits reserved for the exponent field is:

   (a) 5

   (b) 8

   (c) 11

   (d) 23

3. The bias used for the exponent in IEEE 754 binary32 is:

   (a) 127

   (b) 128

   (c) 255

   (d) 1023

4. The 32-bit pattern `0x7F800000` (sign = 0, exponent all ones, mantissa all zeros) represents:

   (a) +∞

   (b) NaN

   (c) the largest normalised number

   (d) +0

EXERCISE 8 **True or false**

State whether each statement is true or false, and justify your answer in one or two sentences.

1. Every decimal fraction with a finite number of decimal digits can be represented exactly in binary.
2. In IEEE 754 binary32, the leading 1 of the normalised significand is not stored; it is implicit.
3. Fixed-point arithmetic is generally faster than floating-point arithmetic on processors without a floating-point unit.
4. In floating-point arithmetic, $(a + b) + c$ is always equal to $a + (b + c)$.
5. The value 0.1 can be represented exactly in IEEE 754 binary32.

EXERCISE 9 **Key definitions**

Give a precise definition of each of the following terms, as used in the representation of fractional numbers:

1. fixed-point representation (Qm.n format);
2. normalised significand (mantissa);
3. exponent bias;
4. denormalised (subnormal) number;
5. machine epsilon;
6. NaN (Not a Number).

EXERCISE 10 **Direct calculations**

1. Convert the following decimal values to binary (fractional part on at most 8 bits): 0.5; 0.625; 3.75; 0.1 (give the first 8 fractional bits).
2. A Q4.4 unsigned fixed-point register contains the pattern `0110.1000`. What decimal value does it represent?
3. In IEEE 754 binary32, what is the decimal value of the biased exponent field `10000010`?
4. How many distinct values can be represented exactly between two consecutive

powers of two, 2^e and 2^{e+1}, in binary32?

12.2 I practise

EXERCISE 11 **From decimal to machine formats**

Convert the decimal value 0.625 to:

1. Q8.8 fixed-point binary (unsigned);
2. IEEE 754 single-precision (binary32), giving the sign bit, the biased exponent and the 23-bit mantissa.

Show all steps of both conversions, then write the final 32-bit pattern in hexadecimal.

EXERCISE 12 **Decoding a 32-bit pattern**

Given the 32-bit pattern `0xC1480000`:

1. split the pattern into sign, exponent and mantissa fields;
2. compute the true (unbiased) exponent;
3. reconstruct the significand and deduce the decimal value of the represented number;
4. state whether the number is normalised, denormalised, infinite or NaN, justifying your answer.

EXERCISE 13 **Mobile money in fixed point**

A mobile money system stores account balances as unsigned Q16.16 fixed-point values (16 integer bits, 16 fractional bits).

1. Determine the maximum balance representable, in FCFA.
2. Determine the smallest non-zero increment (the resolution) of a balance.
3. Explain why this format is well suited to money, and give one limitation of the format for a large financial institution.
4. The balance `0x0001F4A0` is read from the database. What amount in FCFA does it represent?

EXERCISE 14 **Catastrophic loss of precision**

Consider the addition of 1.0×10^{10} and 1.0×10^{-10} in IEEE 754 binary32.

1. Give the approximate value of machine epsilon for binary32 and explain its meaning.
2. Show, by aligning the exponents, why the small operand is completely lost in the sum.
3. Propose a method to mitigate this kind of precision loss when summing a long list of values of very different magnitudes.

12.3 I prepare for the GCE

EXERCISE 15 **Fixed point versus floating point — exam style**

A Yaoundé software company develops an embedded controller for a water-pumping station. The processor has no floating-point unit. Sensor readings (water level in metres) range from 0.00 m to 12.75 m and must be stored with a resolution of at least 0.01 m.

1. Define fixed-point representation and floating-point representation, highlighting the role of the exponent in the latter. (4 marks)
2. Propose an unsigned Qm.n fixed-point format suitable for the sensor readings, justifying the number of integer and fractional bits chosen. (4 marks)
3. Encode the reading 9.375 m in your proposed format, showing all steps. (3 marks)
4. Give two advantages and two disadvantages of fixed-point representation compared with IEEE 754 floating point in this embedded context. (4 marks)
5. The company later needs to store values from 10^{-6} to 10^{9}. Explain why fixed point is no longer appropriate and why IEEE 754 solves the problem. (5 marks)

EXERCISE 16 **IEEE 754 and a conversion routine — exam style**

1. Describe the structure of an IEEE 754 single-precision (binary32) number: sign, exponent, mantissa, bias, implicit leading bit. (5 marks)
2. Convert the decimal value -13.40625 to binary32, giving the final pattern in hexadecimal. Show every step. (6 marks)
3. A banking application stores amounts in cents as signed 32-bit integers. Write, in clear pseudocode, a routine that converts a signed 32-bit integer amount (in cents) to a binary32 floating-point value in FCFA, and a second routine that converts it back

to cents, ensuring no loss for amounts up to 10^9 FCFA. Justify why binary32 is sufficient for this range. (6 marks)

4. State what happens in IEEE 754 when a computation produces a result larger than the largest representable number, and when 0 is divided by 0. (3 marks)

EXERCISE 17 **Rounding errors in a VAT loop — essay question**

A supermarket in Douala computes the VAT-inclusive total of a sale with the following loop, using IEEE 754 binary32 arithmetic: for each of the 5000 items sold in a promotion, priced at 12 345 FCFA each, the program adds the price to a running total, then multiplies the final total by 1.1925.

1. Explain why the value 12 345 is represented exactly in binary32, whereas 1.1925 is not. (4 marks)
2. Analyse the loop and estimate the order of magnitude of the total rounding error, distinguishing the error due to the accumulation from the error due to the final multiplication. (6 marks)
3. Explain the notions of round-off error, absorption and cancellation, illustrating each with this example or a similar one. (6 marks)
4. Propose an integer-based alternative (amounts stored in cents as integers) and explain why it eliminates these errors. Discuss one situation in which floating point would still be preferable. (4 marks)

13 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple choice questions**

1. Correct answer: (b) 1.5. In **Q8.8** format the binary point sits after 8 integer bits. The pattern `00000001.10000000` gives integer part 1 and fractional part $1 \times 2^{-1} = 0.5$, hence $1 + 0.5 = 1.5$. (Answer (a) confuses the binary point with a decimal point; (c) misreads the fractional bit as 2^{-2}; (d) reads the pattern as a decimal string.)

2. Correct answer: (b) 8. IEEE 754 **binary32** uses 1 sign bit, 8 exponent bits and 23 mantissa bits ($1 + 8 + 23 = 32$). The value 11 is the exponent width of **binary64**, 23 is the mantissa width, and 5 is the exponent width of **binary16** (half precision).

3. Correct answer: (a) 127. The bias of binary32 is $2^{8-1} - 1 = 127$. The value 1023 is the bias of binary64; 255 is the maximum value of the 8-bit field (reserved for $\pm\infty$ and NaN); 128 is a common distractor.

4. Correct answer: (a) $+\infty$. An exponent field of all ones with a zero mantissa encodes infinity; the sign bit 0 makes it $+\infty$. A non-zero mantissa with an all-ones exponent would be **NaN**.

SOLUTION **Exercise 2 — True or false**

1. False. A decimal fraction is exactly representable in binary only if it is a sum of powers of two, i.e. its denominator (in lowest terms) is a power of 2. For example $0.5 = 2^{-1}$ is exact, but $0.1 = 1/10$ has denominator $10 = 2 \times 5$, and its binary expansion $0.00011001100\ldots$ is infinite and repeating.

2. True. Every **normalised** binary number is written $1.f \times 2^e$; since the leading digit is always 1, it is not stored. Only the 23 fractional bits f (the mantissa field) are stored, which gains one free bit of precision.

3. True. **Fixed-point** operations reduce to ordinary integer additions, subtractions and shifts, which any ALU performs in one or a few cycles. Floating-point operations require alignment of exponents, normalisation and rounding; without a dedicated FPU these steps must be emulated in software, which is far slower.

4. False. Floating-point addition is not associative. Because each result is rounded to 24 significant bits, the order of operations matters. Classic counter-example: with $a = 10^{10}$, $b = -10^{10}$, $c = 1$, we get $(a + b) + c = 1$ but $a + (b + c) = 0$, because $b + c$ is rounded back to -10^{10} (the 1 is **absorbed**).

5. False. As in statement 1, 0.1 has an infinite repeating binary expansion, so it is rounded

to the nearest representable binary32 value (approximately 0.10000000149). This is the source of the famous $0.1 + 0.2 \neq 0.3$ phenomenon.

SOLUTION **Exercise 3 — Key definitions**

1. **Fixed-point representation (Qm.n)**: a representation in which a binary word of $m + n$ bits is interpreted with a fixed binary point: the m most significant bits form the integer part and the n least significant bits form the fractional part, weighted $2^{-1}, 2^{-2}, \ldots, 2^{-n}$. The stored integer N represents the value $N / 2^n$ (unsigned case). The point is implicit — it exists only by convention.
2. **Normalised significand (mantissa)**: the factor M in the scientific form $x = \pm M \times 2^e$ with $1 \leq M < 2$, i.e. $M = 1.f$ where f is the fractional part. Normalisation guarantees a unique representation and maximal precision; the leading 1 is implicit in IEEE 754.
3. **Exponent bias**: a constant added to the true exponent so that the stored exponent field is always a non-negative integer, allowing exponents to be compared as unsigned integers. In binary32 the bias is 127: stored exponent $E = e + 127$, so true exponent $e = E - 127$.
4. **Denormalised (subnormal) number**: a number encoded with exponent field $E = 0$ and non-zero mantissa. It is interpreted as $\pm 0.f \times 2^{-126}$ (no implicit leading 1). Subnormals fill the gap between 0 and the smallest normal number, providing gradual underflow.
5. **Machine epsilon**: the gap between 1 and the next representable floating-point number, $\varepsilon = 2^{-23} \approx 1.19 \times 10^{-7}$ in binary32. It bounds the relative rounding error of any single operation: the stored result differs from the exact result by a relative error of at most $\varepsilon / 2$ (round to nearest).
6. **NaN (Not a Number)**: a special IEEE 754 code (exponent all ones, mantissa non-zero) representing an undefined or invalid result, e.g. $0/0$, $\infty - \infty$, $\sqrt{-1}$. NaN propagates through subsequent computations, signalling that an error occurred.

SOLUTION **Exercise 4 — Direct calculations**

1. Conversions to binary (repeated multiplication by 2 for the fractional part):
   - $0.5 = 0.1_2$ (since $0.5 \times 2 = 1.0$).
   - 0.625: $0.625 \times 2 = 1.25 \rightarrow 1$; $0.25 \times 2 = 0.5 \rightarrow 0$; $0.5 \times 2 = 1.0 \rightarrow 1$. Hence $0.625 = 0.101_2$.
   - 3.75: integer part $3 = 11_2$; fractional part 0.75: $0.75 \times 2 = 1.5 \rightarrow 1$; $0.5 \times 2 = 1.0 \rightarrow 1$.

Hence $3.75 = 11.11_2$.

- 0.1: successive doublings give $0, 0, 0, 1, 1, 0, 0, 1, \ldots$, i.e. $0.1 \approx 0.00011001_2$ on 8 fractional bits (the pattern `0011` repeats forever).

2. `0110.1000` in Q4.4: integer part $0110_2 = 6$; fractional part $1000_2 = 2^{-1} = 0.5$. Value: 6.5. (Check: stored integer $01101000_2 = 104$, and $104/2^4 = 6.5$.) ✓
3. $E = 10000010_2 = 130$. True exponent $e = E - 127 = 3$.
4. Between 2^e and 2^{e+1} the significand $1.f$ takes all 2^{23} values of the 23-bit mantissa, with the same exponent. Hence exactly $2^{23} = 8\,388\,608$ distinct values (including 2^e, excluding 2^{e+1}), evenly spaced by 2^{e-23}.

SOLUTION **Exercise 5 — From decimal to machine formats**

(a) Q8.8 fixed point. Multiply by $2^8 = 256$: $0.625 \times 256 = 160$. Convert 160 to 16-bit binary: $160 = 128 + 32 = 2^7 + 2^5$, so

$$160 = 0000000010100000_2.$$

Reading with the point after 8 bits: `00000000.10100000`, i.e. fractional bits $2^{-1} + 2^{-3} = 0.5 + 0.125 = 0.625$. ✓

(b) IEEE 754 binary32.

1. Sign: the number is positive, so $S = 0$.
2. Binary: $0.625 = 0.101_2$.
3. Normalise: $0.101_2 = 1.01_2 \times 2^{-1}$. True exponent $e = -1$.
4. Biased exponent: $E = e + 127 = 126 = 01111110_2$.
5. Mantissa: drop the implicit leading 1, keep $f = 01$ padded with zeros to 23 bits: `01000000000000000000000`.

Assembling $S \mid E \mid f$:

`0 01111110 01000000000000000000000`

Grouping by nibbles: `0011 1111 0010 0000 0000 0000 0000 0000`, hence the hexadecimal pattern

$$\boxed{\texttt{0x3F200000}}.$$

SOLUTION **Exercise 6 — Decoding a 32-bit pattern**

(a) Field splitting. `0xC1480000` = `1100 0001 0100 1000 0000 0000 0000 0000`$_2$.

- Sign bit: $S = 1$ (negative number).

- Exponent field: $E = 10000010_2$.
- Mantissa field: $f = 10010000000000000000000_2$.

(b) True exponent. $E = 130$, so $e = E - 127 = 3$.
(c) Significand and value. Since $E \neq 0$ and $E \neq 255$, the number is normalised with implicit leading 1:

$$M = 1.f = 1 + 2^{-1} + 2^{-4} = 1 + 0.5 + 0.0625 = 1.5625.$$

Value: $x = -M \times 2^e = -1.5625 \times 2^3 = -1.5625 \times 8 = \boxed{-12.5}$.
Check: $12.5 = 1100.1_2 = 1.1001_2 \times 2^3$, mantissa 1001 followed by zeros — consistent.
(d) Classification. The exponent field is neither all zeros (00000000) nor all ones (11111111): $E = 130$ is an ordinary value, so the number is a normalised finite number. It is not denormalised (which requires $E = 0$), not infinite and not NaN (which require $E = 255$).

SOLUTION **Exercise 7 — Mobile money in fixed point**

(a) Maximum balance. A Q16.16 unsigned register holds 32 bits; the maximum stored integer is $2^{32} - 1$, and the value is the stored integer divided by 2^{16}:

$$V_{\max} = \frac{2^{32} - 1}{2^{16}} = 2^{16} - 2^{-16} \approx 65\,535.99998 \text{ FCFA}.$$

(b) Resolution. The smallest non-zero increment is one unit of the least significant fractional bit:

$$\Delta = 2^{-16} = \frac{1}{65\,536} \approx 0.0000153 \text{ FCFA},$$

far finer than one centime — the format is effectively exact for money.
(c) Suitability and limitation. Suitability: balances are sums and differences of amounts; in fixed point these are exact integer additions (no rounding error, no absorption), and arithmetic is fast even on cheap processors without an FPU. Limitation: the range is capped at about 65 536 FCFA, which is too small for a large institution handling corporate accounts or aggregated totals of millions of FCFA; a wider format (e.g. Q32.32 in 64 bits) would be needed.
(d) Decoding $0x0001F4A0$. The stored integer is $0x0001F4A0 = 1\times16^4 + 15\times16^3 + 4\times16^2 + 10\times16 = 65\,536 + 61\,440 + 1\,024 + 160 = 128\,160$. Hence

$$V = \frac{128\,160}{65\,536} = 1.9559\ldots$$

Compute exactly: $128\,160/65\,536 = 1 + 62\,624/65\,536 \approx 1.95593$ FCFA. So the balance is approximately $\boxed{1.956 \text{ FCFA}}$ (about 1 FCFA 96 centimes).

SOLUTION **Exercise 8 — Catastrophic loss of precision**

(a) Machine epsilon. For binary32, $\varepsilon = 2^{-23} \approx 1.19 \times 10^{-7}$. It is the gap between 1.0 and the next representable number; equivalently, any real number x is stored with a relative error of at most about $\varepsilon/2 \approx 6 \times 10^{-8}$ — roughly 7 significant decimal digits.
(b) Alignment. Write both operands with the same exponent. $1.0 \times 10^{10} \approx 2^{33.2}$, so its least significant mantissa bit has weight about $2^{33} \times 2^{-23} = 2^{10} \approx 10^3$. Aligning 10^{-10} to the exponent of 10^{10} shifts its significand more than 43 binary places to the right — far beyond the 24-bit significand. Every bit of the small operand falls off the end of the adder, so

$$\mathrm{fl}\left(10^{10} + 10^{-10}\right) = 10^{10}.$$

The small operand is completely absorbed: the sum is bit-for-bit identical to the large operand.
(c) Mitigation. Sort the values by increasing absolute magnitude and sum the small ones first, so partial sums of small terms can grow to a comparable size before meeting the large terms. More robustly, use compensated (Kahan) summation: keep a running compensation variable that recovers the low-order bits lost at each addition, reducing the accumulated error from $O(n\varepsilon)$ to nearly $O(\varepsilon)$.

SOLUTION **Exercise 9 — Fixed point versus floating point (exam style)**

(a) Definitions (4 marks). Fixed point: a binary word is split by convention into m integer bits and n fractional bits; the binary point is implicit and never moves, so the value is $N/2^n$ for the stored integer N. Floating point: a number is stored in normalised scientific form $\pm 1.f \times 2^e$; the exponent e (stored biased) records the position of the binary point, allowing the point to "float" and giving a very wide dynamic range. Mark scheme: 1 mark per correct definition, 1 mark for the role of the exponent, 1 mark for the normalised form/implicit point.
(b) Format choice (4 marks). Range needed: up to 12.75 m, so 4 integer bits suffice ($2^4 = 16 > 12.75$). Resolution: at least 0.01 m, so we need $2^{-n} \le 0.01$, i.e. $2^n \ge 100$; $n = 7$ gives $2^{-7} = 0.0078125 \le 0.01$. Proposed format: unsigned Q4.7 (11 bits; fits in a 16-bit register). Examiner expects: justification of both bit counts by inequalities, not just a guess.
(c) Encoding 9.375 m (3 marks). Integer part: $9 = 1001_2$. Fractional part: $0.375 = 0.25 + 0.125 = 2^{-2} + 2^{-3} = 0.011_2$. So $9.375 = 1001.011_2$. In Q4.7, pad the fraction to 7 bits: `1001.0110000`. Check: stored integer $10010110000_2 = 1200$, and $1200/2^7 = 1200/128 = 9.375$. ✓
(d) Advantages and disadvantages (4 marks). Advantages: (i) arithmetic is exact integer arithmetic — fast on a processor with no FPU; (ii) representation is exact for the chosen resolution — no rounding surprises, and memory usage is small. Disadvantages: (i) very limited range — overflow if the level ever exceeds 16 m; (ii) fixed resolution — precision cannot adapt, and multiplication/division require manual rescaling (risk of overflow or loss of bits).

(e) Why fixed point fails (5 marks). The new range spans 10^{-6} to 10^{9}, a dynamic range of $10^{15} \approx 2^{50}$. A fixed-point format covering both ends would need about 30 fractional bits and 30 integer bits — over 60 bits — and would still waste precision everywhere. IEEE 754 solves this because the exponent lets the binary point float: binary32 covers roughly 10^{-38} to 10^{38} with about 7 significant digits at every magnitude, so both 10^{-6} and 10^{9} are represented with full relative precision in only 32 bits. Examiner expects: the dynamic-range argument quantified, and the role of the exponent explicitly stated.

SOLUTION **Exercise 10 — IEEE 754 and a conversion routine (exam style)**

(a) Structure of binary32 (5 marks). 32 bits split as: 1 sign bit S; 8 exponent bits storing $E = e + 127$ (bias 127, so true exponents range from -126 to +127 for normal numbers); 23 mantissa bits storing the fraction f of the normalised significand $1.f$ — the leading 1 is implicit and not stored. Value: $x = (-1)^S \times 1.f \times 2^{E-127}$. Special codes: $E = 0$ for zero/subnormals, $E = 255$ for $\pm\infty$ (if $f = 0$) and NaN (if $f \neq 0$).

(b) Conversion of -13.40625 (6 marks).

1. Sign: negative, so $S = 1$.
2. Integer part: $13 = 1101_2$.
3. Fractional part: $0.40625 \times 2 = 0.8125 \to 0$; $0.8125 \times 2 = 1.625 \to 1$; $0.625 \times 2 = 1.25 \to 1$; $0.25 \times 2 = 0.5 \to 0$; $0.5 \times 2 = 1.0 \to 1$. So $0.40625 = 0.01101_2$.
4. $13.40625 = 1101.01101_2 = 1.10101101_2 \times 2^3$ (point shifted 3 places left).
5. Biased exponent: $E = 3 + 127 = 130 = 10000010_2$.
6. Mantissa (23 bits): `10101101000000000000000`.

Pattern: `1 10000010 10101101000000000000000 = 1100 0001 0101 0110 1000 0000 0000 0000`, i.e. `0xC1568000`.

(c) Conversion routines (6 marks).

```
FUNCTION CentsToFcfa(c : INTEGER) : REAL
  RETURN c / 100.0   // binary32 division
END FUNCTION

FUNCTION FcfaToCents(x : REAL) : INTEGER
  RETURN ROUND(x * 100.0)   // round to nearest cent
END FUNCTION
```

Justification. Binary32 represents all integers exactly up to $2^{24} \approx 1.68 \times 10^{7}$. The maximum amount is 10^{9} FCFA = 10^{11} cents, which far exceeds 2^{24}. Therefore, for the full range up to 10^{9} FCFA, the routines are not guaranteed to give exact round-trips using binary32; the

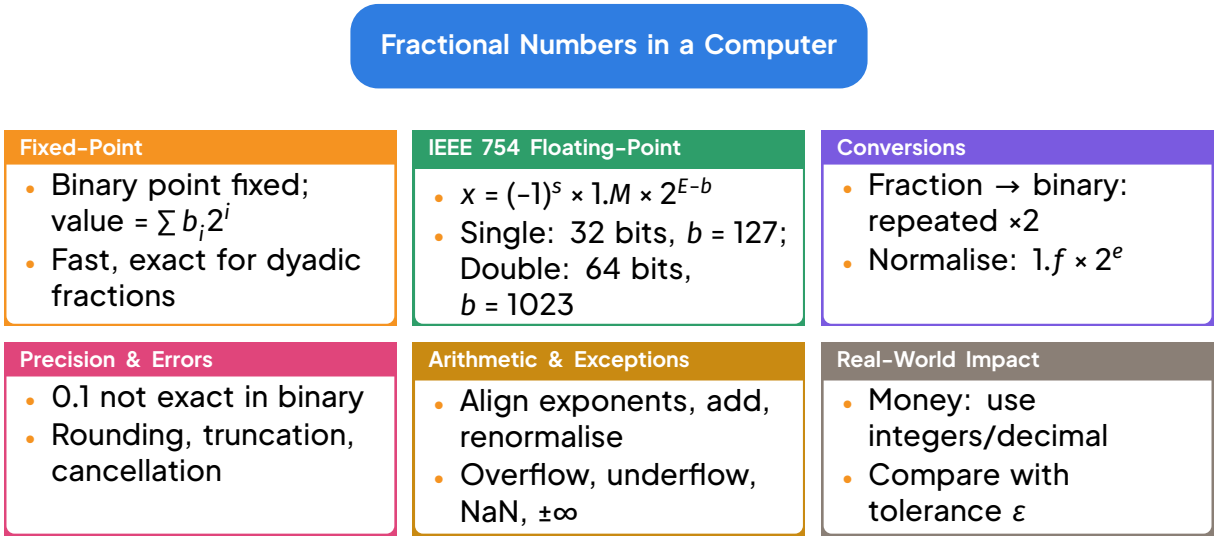
absolute error on 10^{11} cents can reach $10^{11} \times 2^{-24} \approx 6\,000$ cents. The routines are safe only for amounts below about 2^{24} cents (≈ 167 772 FCFA). For the full specified range, **binary64** (53-bit significand, exact integers to 9×10^{15}) or pure 64-bit integer arithmetic should be used. Mark scheme: 2 marks per routine, 2 marks for a correct, quantified justification of the precision limit.
(d) Exceptions (3 marks). A result larger than the largest representable number (≈ 3.4×10^{38}) produces overflow: the result becomes $+\infty$ (or $-\infty$) and the overflow flag is raised. The computation $0/0$ is invalid: it produces NaN and raises the invalid-operation flag.

SOLUTION **Exercise 11 — Rounding errors in a VAT loop (essay)**

(a) Exactness (4 marks). 12 345 is an integer well below $2^{24} = 16\,777\,216$, so it fits exactly in the 24-bit significand of binary32 ($12\,345 = 11000000111001_2$, 14 bits). By contrast $1.1925 = 11925/10000 = 477/400$ in lowest terms; the denominator $400 = 2^4 \times 5^2$ contains a factor 5^2, so its binary expansion is infinite and repeating — it must be rounded to 24 bits and is not represented exactly.
(b) Error analysis (6 marks). Accumulation error: each addition of 12 345 to the running total is rounded with a relative error up to $\varepsilon/2 \approx 6 \times 10^{-8}$. As the total grows toward $5000 \times 12\,345 = 61\,725\,000 \approx 6.2 \times 10^7$, the absolute rounding per step grows to about $6.2 \times 10^7 \times 6 \times 10^{-8} \approx 3.7$ FCFA per addition. Summed over 5000 steps, the worst-case accumulated error is of order $5000 \times 3.7 \approx 1.8 \times 10^4$ FCFA; statistically (random signs) it behaves more like $\sqrt{5000} \times 3.7 \approx 260$ FCFA. Note also that $6.2 \times 10^7 > 2^{24}$, so near the end the total is no longer an exact integer. Multiplication error: the final product by the inexact constant 1.1925 adds one more relative error of order 6×10^{-8}, i.e. about $7.4 \times 10^7 \times 6 \times 10^{-8} \approx 4$ FCFA — negligible compared with the accumulation. Conclusion: the dominant error comes from the 5000 rounded additions, of order 10^2 to 10^4 FCFA.
(c) Round-off, absorption, cancellation (6 marks). Round-off error: the difference between an exact result and its stored, rounded value — here each `total + price` is rounded to 24 significant bits. Absorption: when adding quantities of very different magnitudes, the small one is partly or wholly lost; here, once the total exceeds 2^{24}, adding 12 345 loses its low-order bits — the unit digits of the price are absorbed by the large total. Cancellation: subtracting two nearly equal numbers eliminates the accurate leading digits and leaves mostly rounding noise; e.g. if the program later computed a discount as `total_after - total_before` with two nearly equal totals, the difference would have very few reliable digits.
(d) Integer alternative (4 marks). Store every amount in centimes (or in whole FCFA here, since prices are integers) as a 64-bit integer: the loop becomes an exact integer accumulation $5000 \times 12\,345 = 61\,725\,000$ — no rounding at all. The VAT can be applied as integer arithmetic: `total * 11925 / 10000` with rounding to the nearest FCFA, a single, controlled rounding. Errors disappear because integer addition is exact. When floating point is still preferable: computations involving a huge dynamic range or genuine fractional physical quantities — e.g. compound-interest projections, scientific simulations,

or exchange-rate conversions spanning many orders of magnitude — where fixed scaling is impractical and relative precision is what matters.

14 SUMMARY SHEET



Mind map of the chapter: representing fractional numbers.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- **Fixed-point representation**: the binary point occupies a fixed position; with n integer bits and m fractional bits, a number is $x = \sum_{i=-m}^{n-1} b_i 2^i$.
- **Floating-point representation**: a number is written in normalised form $x = (-1)^s \times 1.M \times 2^e$, like scientific notation in base 2.
- **Sign bit** s: 0 for positive, 1 for negative.
- **Biased exponent**: $E = e + b$, where the bias is $b = 127$ (single) or $b = 1023$ (double).
- **Mantissa** M: the fractional part after the leading (hidden) 1.
- **Dyadic fraction**: a fraction whose denominator is a power of 2; only these are exactly representable in binary.

Formulas and facts to know

- IEEE 754 single precision (32 bits): 1 sign bit, 8 exponent bits, 23 mantissa bits; bias 127; about 7 significant decimal digits.
- IEEE 754 double precision (64 bits): 1 sign bit, 11 exponent bits, 52 mantissa bits; bias 1023; about 15–16 significant decimal digits.
- Value decoded: $x = (-1)^s \times \left(1 + \sum_{i=1}^{k} m_i 2^{-i}\right) \times 2^{E-b}$.

- Special patterns: $E = 0, M = 0 \Rightarrow \pm 0$; $E = 0, M \neq 0 \Rightarrow$ denormalised number; $E =$ all 1, $M = 0 \Rightarrow \pm\infty$; $E =$ all 1, $M \neq 0 \Rightarrow$ NaN.
- Fixed-point range with $n + m$ bits (unsigned): $[0,\ 2^n - 2^{-m}]$, step (resolution) 2^{-m}.

Methods

1. Decimal fraction → binary: multiply the fractional part repeatedly by 2; record the integer parts (0 or 1) in order; stop when the fraction is 0 or the required number of bits is reached. Example: $0.625 \rightarrow 0.101_2$; $0.1 \rightarrow 0.00011001100\ldots_2$ (never terminates!).
2. Decimal → IEEE 754 single: convert to binary; normalise as $1.f \times 2^e$; compute $E = e + 127$ in binary; write $s \mid E \mid M$ with $M = f$ padded to 23 bits.
3. IEEE 754 → decimal: read s, compute $e = E - 127$, rebuild $1.M$, shift the point by e places, convert, apply the sign.
4. Floating-point addition: align exponents (shift the smaller mantissa right), add, renormalise, round.

Common mistakes

- Forgetting the hidden leading 1 of the normalised mantissa.
- Confusing the exponent e with the stored (biased) exponent E.
- Believing 0.1, 0.2, 0.3 are exact in binary — they are not.
- Testing equality of floats with = instead of $|x - y| < \varepsilon$.
- Ignoring overflow (result too large → ∞) and underflow (result too small → 0).
- Using floats for money (e.g. FCFA amounts): accumulate integer francs or use decimal types.

GCE tips

- Show every step of a conversion: examiners award method marks.
- Always state the bias you use and check your answer by converting back.
- For "explain why $0.1 + 0.2 \neq 0.3$", argue with the infinite binary expansion of 0.1 and 0.2 and the finite mantissa.
- Draw the 32-bit layout (s / E / M) clearly with the number of bits under each field.
- Manage your time: a full IEEE 754 conversion takes about 5 minutes when practised.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+